Valence Alignment in Human-AI Story Co-Creation

Analysis of Field Data and AI-AI Baseline Comparison

PENPAL Project

2026-02-22

Contents

0.1 Summary . . . 2

1 1. Data Loading and Preprocessing . . . 2
1.1 1.1 Load Data . . . 2
1.2 1.2 Data Preprocessing . . . 3
1.3 1.3 Compute Lagged Variables and Cosine . . . 4

2 2. Descriptive Statistics . . . 6
2.1 2.1 Token Count Statistics . . . 6
2.2 2.2 Valence Distribution Statistics . . . 6

3 3. Valence Distribution Analysis . . . 7
3.1 3.1 Valence Distribution Difference Test . . . 7
3.2 3.2 Combined Valence Distribution Plot . . . 8

4 4. Valence Alignment Analysis . . . 11
4.1 4.1 Per-Story Correlation (Same Turn) . . . 11
4.2 4.2 Per-Story Correlation (Previous Turn) . . . 11
4.3 4.3.1 All turns story correlation (same turn) . . . 12
4.4 4.3.2 All turns story correlation (Previous Turn) . . . 13
4.5 4.3.3 Probe Sentiment Narrative Development . . . 14
4.6 4.4.1 Alignment Comparison Plot . . . 16
4.7 4.4.2 Comparison: Per-Story vs Aggregate Correlations . . . 18
4.8 4.6 Statistical Tests: Alignment Differences . . . 24
4.9 Cosine Distribution Difference Test . . . 26
4.10 Combined Alignment Plot . . . 29
4.11 Valence Distance Difference Test . . . 31
4.12 4.7 Alignment Duration and Stability . . . 33
4.12.1 4.7.1 Visualization: Alignment Frequency (Stability) . . . 35
4.12.2 4.7.2 Visualization: Duration Distribution (Survival Analysis) . . . 36

5 6. Summary and Conclusions . . . 38
5.1 6.1 Key Findings Summary . . . 38

6 7. Supplementary: Interaction Effects . . . 38
6.1 7.1 Starter Interaction Effects . . . 38
6.1.1 7.1.1 Same-Turn Alignment by Starter (Removed for now) . . . 38
6.1.2 7.1.2 Previous-Turn Alignment by Starter (Removed for now) . . . 39
6.1.3 7.1.3 Per-Story Correlations by Starter (Removed for now) . . . 39

6.2 7.2 LLM Type Interaction Effects . . . 39
6.2.1 7.2.1 Same-Turn Alignment by LLM Type . . . 39
6.2.2 7.2.2 Previous-Turn Alignment by LLM Type . . . 41
6.2.3 7.2.3 Per-Story Correlations by LLM Type . . . 42
6.3 7.3 Valence Distribution by Conditions . . . 43
6.3.1 7.3.1 Valence by Starter . . . 43
6.3.2 7.3.2 Valence by LLM Type . . . 43
6.4 8. Session Information . . . 44

0.1 Summary

- Emotional alignment is asymmetric: the model reliably aligns to the user (User → AI), while humans do not mirror the preceding AI tone.
- Field data shows greater variance in valence and weaker overall coordination compared to the simulated baseline.
- Overall, affective dynamics in the field are primarily model-driven rather than mutually convergent.

1 1. Data Loading and Preprocessing

1.1 1.1 Load Data

```r
# Load dyadic sentiment scores

df_field <- read_parquet(here("data/TEXT/processed/dyadic_sentiment_scores.parquet"))
df_simulated <- read_parquet(here("data/Berlin/processed/dyadic_sentiment_scores_simulated.parquet"))

# Display structure
cat("Field data dimensions:", dim(df_field), "\n")
```

```
## Field data dimensions: 870 17
```

```r
cat("Simulated data dimensions:", dim(df_simulated), "\n")
```

```
## Simulated data dimensions: 30 12
```

```r
# Show column names
cat("\nField data columns:\n")
```

```
##
## Field data columns:
```

```r
print(names(df_field))
```

```
##  [1] "timestamp"                 "user"
##  [3] "ai"                        "conversation_id"
##  [5] "respondent_id"             "interaction_count"
##  [7] "llm_type"                  "edit_distance"
##  [9] "turn"                      "starter"
## [11] "user_embedding"            "ai_embedding"
## [13] "pct_turn"                  "user_sentiment_score"
## [15] "ai_sentiment_score"        "user_sentiment_projection"
## [17] "ai_sentiment_projection"
```

```r
cat("\nSimulated data columns:\n")
```

```
##
## Simulated data columns:
```

```r
print(names(df_simulated))
```

```
##  [1] "turn"                 "user"                 "ai"
##  [4] "client_id"            "workshop_id"          "conversation_id"
##  [7] "timestamp"            "user_embedding"       "ai_embedding"
## [10] "pct_turn"             "user_sentiment_score" "ai_sentiment_score"
```

1.2 1.2 Data Preprocessing

```r
# Preprocess field data
df_field <- df_field %>%
  rename(
    ai_valence = ai_sentiment_projection,
    user_valence = user_sentiment_projection
  ) %>%
  mutate(
    # Token counts
    user_token_count = stri_count_words(user),
    ai_token_count = stri_count_words(ai),
    token_ratio = if_else(ai_token_count == 0, NA_real_,
                          user_token_count / ai_token_count),

    # Time parsing
    timestamp = ymd_hms(timestamp),

    # Data source label
    data_source = "Field"
  ) %>%
  # Ensure conversation_id is a character/factor
  mutate(conversation_id = as.character(conversation_id)) %>%
  filter(turn > 1, turn < 10)


# Preprocess simulated data
df_simulated <- df_simulated %>%
  rename(
    ai_valence = ai_sentiment_score,
    user_valence = user_sentiment_score
  ) %>%
  mutate(
    # Token counts
    user_token_count = stri_count_words(user),
    ai_token_count = stri_count_words(ai),
    token_ratio = if_else(ai_token_count == 0, NA_real_,
                          user_token_count / ai_token_count),

    # Time parsing (if available)
    timestamp = if_else(!is.na(timestamp), ymd_hms(timestamp), as.POSIXct(NA)),

    # Data source label
    data_source = "Simulated"
  ) %>%
  mutate(conversation_id = as.character(conversation_id))
```

```r
# Summary statistics
cat("\n=== Field Data Summary ===\n")
```

```
##
## === Field Data Summary ===
```

```r
df_field %>%
  summarise(
    n_conversations = n_distinct(conversation_id),
    n_turns = n(),
    mean_turns_per_story = n() / n_distinct(conversation_id),
    mean_user_valence = mean(user_valence, na.rm = TRUE),
    mean_ai_valence = mean(ai_valence, na.rm = TRUE)
  ) %>%
  print()
```

```
## # A tibble: 1 x 5
##   n_conversations n_turns mean_turns_per_story mean_user_valence mean_ai_valence
##             <int>   <int>                <dbl>             <dbl>           <dbl>
## 1              87     696                    8           -0.0891         0.00381
```

```r
cat("\n=== Simulated Data Summary ===\n")
```

```
##
## === Simulated Data Summary ===
```

```r
df_simulated %>%
  summarise(
    n_conversations = n_distinct(conversation_id),
    n_turns = n(),
    mean_turns_per_story = n() / n_distinct(conversation_id),
    mean_user_valence = mean(user_valence, na.rm = TRUE),
    mean_ai_valence = mean(ai_valence, na.rm = TRUE)
  ) %>%
  print()
```

```
## # A tibble: 1 x 5
##   n_conversations n_turns mean_turns_per_story mean_user_valence mean_ai_valence
##             <int>   <int>                <dbl>             <dbl>           <dbl>
## 1               6      30                    5            -0.407          -0.296
```

1.3 1.3 Compute Lagged Variables and Cosine

```r
# Add lagged AI valence for field data
# Note: lag() within group_by ensures no cross-story contamination
# Turn 1 of each story will have NA for lagged values (correct behavior)

df_field <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_valence_lag1 = lag(ai_valence, n = 1),
    user_valence_lag1 = lag(user_valence, n = 1),
  ) %>%
  ungroup()
```

```r
cosine_sim <- function(x, y) {
  if (is.null(x) || is.null(y) || any(is.na(x)) || any(is.na(y))) {
    return(NA_real_)
  }
  return(sum(x * y) / (sqrt(sum(x^2)) * sqrt(sum(y^2))))
}

df_field <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_embedding_prev = lag(ai_embedding),
    user_embedding_prev = lag(user_embedding)
  ) %>%
  rowwise() %>%
  mutate(
    cosine_sim_same = cosine_sim(user_embedding, ai_embedding),
    cosine_sim_prev = cosine_sim(user_embedding, ai_embedding_prev),
    cosine_sim_user_self = cosine_sim(user_embedding, user_embedding_prev),
    cosine_sim_ai_self = cosine_sim(ai_embedding, ai_embedding_prev)
  ) %>%
  ungroup()

# Add lagged AI valence for simulated data
df_simulated <- df_simulated %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_valence_lag1 = lag(ai_valence, n = 1),
    user_valence_lag1 = lag(user_valence, n = 1)
  ) %>%
  ungroup()

# Verify no cross-story contamination
cat("=== Lag Variable Verification ===\n")
```

```
## === Lag Variable Verification ===
```

```r
cat("Field data - Turn 1 with non-NA lag (should be 0):",
    sum(df_field$turn == 1 & !is.na(df_field$ai_valence_lag1)), "\n")
```

```
## Field data - Turn 1 with non-NA lag (should be 0): 0
```

```r
cat("Simulated data - Turn 1 with non-NA lag (should be 0):",
    sum(df_simulated$turn == 1 & !is.na(df_simulated$ai_valence_lag1)), "\n")
```

```
## Simulated data - Turn 1 with non-NA lag (should be 0): 0
```

2 2. Descriptive Statistics

2.1 2.1 Token Count Statistics

```
# Field data token statistics
token_stats_field <- df_field %>%
  summarise(
    mean_user_tokens = mean(user_token_count, na.rm = TRUE),
    sd_user_tokens = sd(user_token_count, na.rm = TRUE),
    mean_ai_tokens = mean(ai_token_count, na.rm = TRUE),
    sd_ai_tokens = sd(ai_token_count, na.rm = TRUE),
    mean_token_ratio = mean(token_ratio, na.rm = TRUE)
  )

cat("=== Field Data Token Statistics ===\n")
```

```
## === Field Data Token Statistics ===
```

```
print(token_stats_field)
```

```
## # A tibble: 1 x 5
##   mean_user_tokens sd_user_tokens mean_ai_tokens sd_ai_tokens mean_token_ratio
##              <dbl>          <dbl>          <dbl>        <dbl>            <dbl>
## 1             25.4           5.28           29.1         7.45            0.913
```

```
# Simulated data token statistics
token_stats_sim <- df_simulated %>%
  summarise(
    mean_user_tokens = mean(user_token_count, na.rm = TRUE),
    sd_user_tokens = sd(user_token_count, na.rm = TRUE),
    mean_ai_tokens = mean(ai_token_count, na.rm = TRUE),
    sd_ai_tokens = sd(ai_token_count, na.rm = TRUE),
    mean_token_ratio = mean(token_ratio, na.rm = TRUE)
  )

cat("\n=== Simulated Data Token Statistics ===\n")
```

```
##
## === Simulated Data Token Statistics ===
```

```
print(token_stats_sim)
```

```
## # A tibble: 1 x 5
##   mean_user_tokens sd_user_tokens mean_ai_tokens sd_ai_tokens mean_token_ratio
##              <dbl>          <dbl>          <dbl>        <dbl>            <dbl>
## 1             26.0           2.17           41.1         3.73            0.636
```

2.2 2.2 Valence Distribution Statistics

```
# Combined statistics
valence_stats <- bind_rows(
  df_field %>%
    pivot_longer(cols = c(user_valence, ai_valence),
                 names_to = "agent", values_to = "valence") %>%
    mutate(agent = str_remove(agent, "_valence")) %>%
    group_by(data_source, agent) %>%
```

```r
    summarise(
      mean_valence = mean(valence, na.rm = TRUE),
      sd_valence = sd(valence, na.rm = TRUE),
      median_valence = median(valence, na.rm = TRUE),
      .groups = "drop"
    ),

  df_simulated %>%
    pivot_longer(cols = c(user_valence, ai_valence),
                 names_to = "agent", values_to = "valence") %>%
    mutate(agent = str_remove(agent, "_valence")) %>%
    group_by(data_source, agent) %>%
    summarise(
      mean_valence = mean(valence, na.rm = TRUE),
      sd_valence = sd(valence, na.rm = TRUE),
      median_valence = median(valence, na.rm = TRUE),
      .groups = "drop"
    )
)

print(valence_stats)
```

```
## # A tibble: 4 x 5
##   data_source agent mean_valence sd_valence median_valence
##   <chr>       <chr>        <dbl>      <dbl>          <dbl>
## 1 Field       ai         0.00381      0.397        -0.0212
## 2 Field       user      -0.0891       0.403        -0.0895
## 3 Simulated   ai        -0.296        0.681        -0.487
## 4 Simulated   user      -0.407        0.529        -0.504
```

```r
# Save to CSV
write_csv(valence_stats, file.path(TABLES_DIR, "valence_descriptive_stats.csv"))
```

3 3. Valence Distribution Analysis

3.1 3.1 Valence Distribution Difference Test

```r
df_field_long <- df_field %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  mutate(agent = recode(agent,
                        user_valence = "User",
                        ai_valence = "LLM"),
         agent = factor(agent, levels = c("User", "LLM"))
         )



df_field_cosine_long <- df_field %>%
  pivot_longer(cols = c(cosine_sim_same, cosine_sim_prev),
               names_to = "direction", values_to = "cosine_sim") %>%
  mutate(direction = recode(direction,
                            cosine_sim_same = "Same",
```

```r
                              cosine_sim_prev = "Prev"),
         direction = factor(direction, levels = c("Same", "Prev"))
         )

df_sim_long <- df_simulated %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  mutate(agent = recode(agent,
                        user_valence = "AI 1",
                        ai_valence = "AI 2"))


# Calculate means for vertical lines
means_field <- df_field_long %>%
  group_by(agent) %>%
  summarise(mean_val = mean(valence, na.rm = TRUE), .groups = "drop")

means_sim <- df_sim_long %>%
  group_by(agent) %>%
  summarise(mean_val = mean(valence, na.rm = TRUE), .groups = "drop")


valence_difference_mixed_model <- lmer(valence ~ agent + (agent | conversation_id), data=df_field_long)
cat("=== Valence Difference Model Coefficients ===\n")
```

```
## === Valence Difference Model Coefficients ===
```

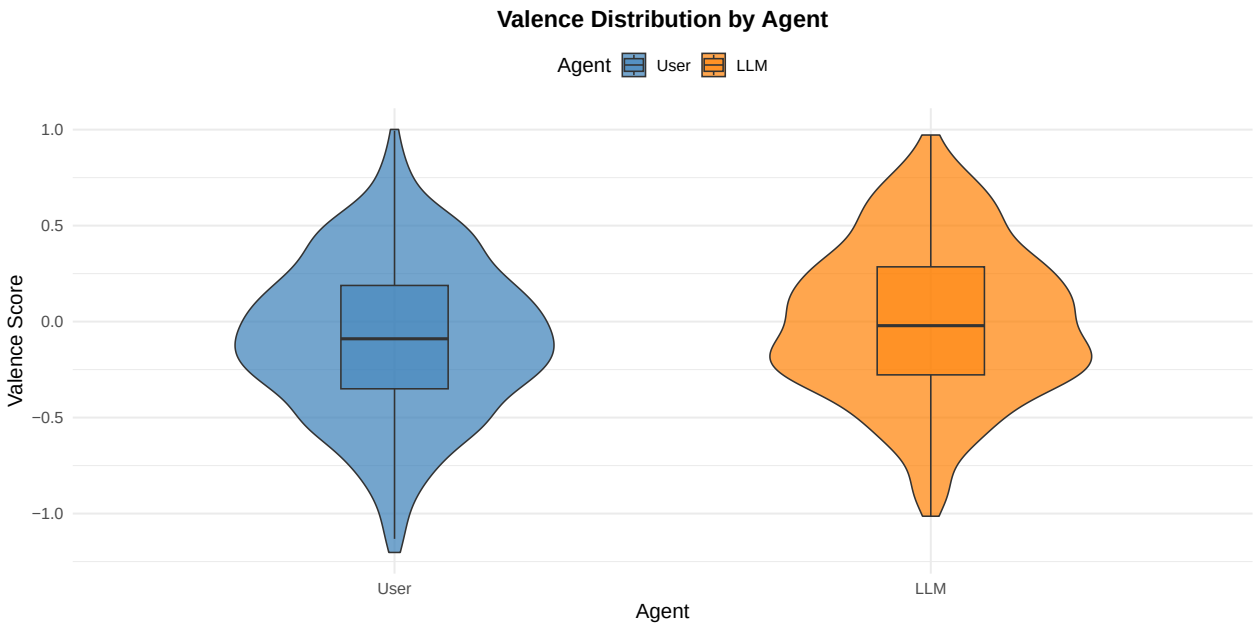
```r
print(round(summary(valence_difference_mixed_model)$coefficients, 4))
```

```
##             Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)  -0.0891     0.0259 86.0015 -3.4428    9e-04
## agentLLM      0.0929     0.0214 86.0007  4.3447    0e+00
```

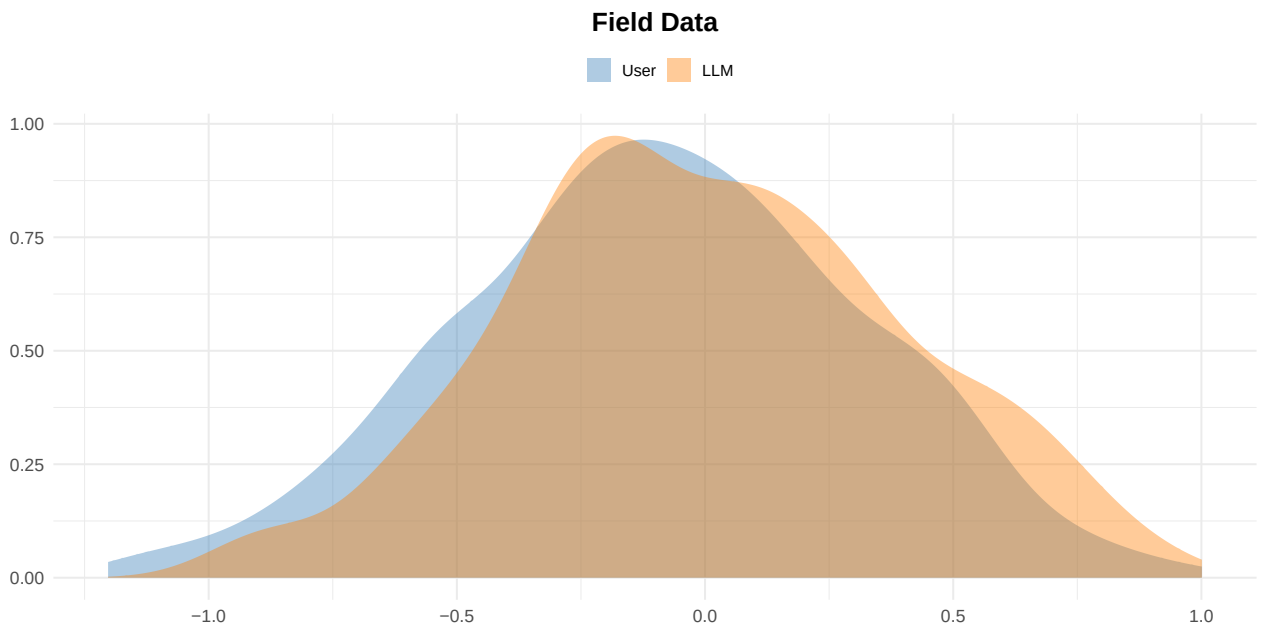
3.2 3.2 Combined Valence Distribution Plot

```r
# Prepare data for plotting

p_distribution_violin <- df_field_long %>%
  ggplot(aes(x = agent, y = valence, fill = agent)) +
  geom_violin(width = 0.6, alpha = 0.7) +
  geom_boxplot(width = 0.2, alpha = 0.5, outlier.shape = NA) +
  scale_fill_manual(values = c("User" = "#377EB8", "LLM" = "#FF7F00"), name = "Agent") +
  scale_color_manual(values = c("User" = "#377EB8", "LLM" = "#FF7F00"), name = "Agent") +
  labs(
    title = "Valence Distribution by Agent",
    x = "Agent",
    y = "Valence Score"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16)
  )
```

```r
print(p_distribution_violin)
```




```r
ggsave(file.path(GRAPHS_DIR, "valence_distribution_comparison_violin.png"),
       p_distribution_violin, width = 12, height = 6, dpi = 300)

# Field data plot
p_field <- ggplot(df_field_long, aes(x = valence, fill = agent)) +
  geom_density(alpha = 0.4, color = NA) +
  scale_fill_manual(values = c("User" = "#377EB8", "LLM" = "#FF7F00"), name = "") +
  scale_color_manual(values = c("User" = "#377EB8", "LLM" = "#FF7F00")) +
  labs(title = "Field Data", x = NULL, y = NULL) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.text = element_text(size = 12)
  )
print(p_field)
```




```
# Simulated data plot
#p_sim <- ggplot(df_sim_long, aes(x = valence, fill = agent)) +
#  geom_density(alpha = 0.4, color = NA) +
#  geom_vline(data = means_sim,
#             aes(xintercept = mean_val, color = agent),
#             linetype = "dashed", linewidth = 0.8, show.legend = FALSE) +
#  scale_fill_brewer(palette = "Set2", name = "") +
#  scale_color_brewer(palette = "Set2") +
#  labs(title = "Simulated Data", x = NULL, y = NULL) +
#  theme(
#    legend.position = "top",
#    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
#    axis.text = element_text(size = 12)
#  )

# Combine plots
#combined_plot <- (p_field) +
#  plot_annotation(
#    title = "Distribution of Valence Scores: Field vs Simulated Data",
#    theme = theme(plot.title = element_text(hjust = 0.5, size = 20, face = "bold"))
#  )
#
# Add axis labels using cowplot
#final_valence_dist <- ggdraw() +
#  draw_plot(combined_plot, 0, 0, 1, 1) +
#  draw_label("Valence Score", x = 0.5, y = 0.02, size = 16) +
#  draw_label("Density", x = 0.02, y = 0.5, angle = 90, size = 16)

#print(final_valence_dist)

#ggsave(file.path(GRAPHS_DIR, "valence_distribution_comparison.png"),
#       final_valence_dist, width = 12, height = 6, dpi = 300)
```

4 4. Valence Alignment Analysis

4.1 4.1 Per-Story Correlation (Same Turn)

```
# Field data: same-turn correlation
cor_field_same <- df_field %>%
  group_by(conversation_id) %>%
  filter(n() >= 3) %>%  # Require at least 3 turns
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Field",
    .groups = "drop"
  )

# Simulated data: same-turn correlation
cor_sim_same <- df_simulated %>%
  group_by(conversation_id) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Simulated",
    .groups = "drop"
  )

# Combine
cor_same_combined <- bind_rows(cor_field_same, cor_sim_same) %>%
  mutate(alignment_type = "Same Turn")

# Summary
cat("=== Same-Turn Alignment Summary ===\n")
```

```
## === Same-Turn Alignment Summary ===
```

```
cor_same_combined %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   data_source mean_rho sd_rho median_rho n_stories
##   <chr>          <dbl>  <dbl>      <dbl>     <int>
## 1 Field          0.232  0.389      0.290        87
## 2 Simulated     -0.246  0.440     -0.314         6
```

4.2 4.2 Per-Story Correlation (Previous Turn)

```r
# Field data: previous-turn correlation (User response to previous AI)
cor_field_prev <- df_field %>%
  group_by(conversation_id) %>%
  filter(!is.na(ai_valence_lag1)) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    data_source = "Field",
    .groups = "drop"
  )

# Simulated data: previous-turn correlation
cor_sim_prev <- df_simulated %>%
  group_by(conversation_id) %>%
  filter(!is.na(ai_valence_lag1)) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    data_source = "Simulated",
    .groups = "drop"
  )

# Combine
cor_prev_combined <- bind_rows(cor_field_prev, cor_sim_prev) %>%
  mutate(alignment_type = "Previous Turn")

# Summary
cat("=== Previous-Turn Alignment Summary ===\n")
```

```
## === Previous-Turn Alignment Summary ===
```

```r
cor_prev_combined %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   data_source mean_rho sd_rho median_rho n_stories
##   <chr>          <dbl>  <dbl>      <dbl>     <int>
## 1 Field         0.0908  0.395      0.123        87
## 2 Simulated     0.533   0.224      0.546         6
```

4.3 4.3.1 All turns story correlation (same turn)

```r
#Violates assumptions since turns are not independent (nested in stories)
cor_field_same_all_turns <- df_field %>%
  summarise(
```

```r
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Field"
  )

cor_sim_same_all_turns <- df_simulated %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Simulated"
  )

cor_same_combined_all_turns <- bind_rows(cor_field_same_all_turns, cor_sim_same_all_turns) %>%
  mutate(alignment_tupe = "Same Turn All Turns")

cat("=== Same-Turn All Turns Alignment Summary ===\n")
```

```
## === Same-Turn All Turns Alignment Summary ===
```

```r
cor_same_combined_all_turns %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   data_source mean_rho sd_rho median_rho n_stories
##   <chr>          <dbl>  <dbl>      <dbl>     <int>
## 1 Field          0.427     NA      0.427         1
## 2 Simulated     -0.110     NA     -0.110         1
```

4.4 4.3.2 All turns story correlation (Previous Turn)

```r
# Violates independence assumption (turns nested in stories)
# Note: use="complete.obs" automatically excludes turn 1 (where ai_valence_lag1 is NA)
# This ensures user_t1 is never paired with ai from previous story

# Verify exclusion of turn 1
cat("=== Previous-Turn Aggregate Correlation ===\n")
```

```
## === Previous-Turn Aggregate Correlation ===
```

```r
cat("Field: Turn 1 observations (excluded):", sum(df_field$turn == 1), "\n")
```

```
## Field: Turn 1 observations (excluded): 0
```

```r
cat("Field: Valid lag observations:", sum(!is.na(df_field$ai_valence_lag1)), "\n")
```

```
## Field: Valid lag observations: 609
```

```r
cat("Simulated: Turn 1 observations (excluded):", sum(df_simulated$turn == 1), "\n")
```

```
## Simulated: Turn 1 observations (excluded): 6
```

```r
cat("Simulated: Valid lag observations:", sum(!is.na(df_simulated$ai_valence_lag1)), "\n\n")
```

```
## Simulated: Valid lag observations: 24
```

```r
cor_field_same_all_turns_prev <- df_field %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    n_valid = sum(!is.na(ai_valence_lag1)),
    data_source = "Field"
  )

cor_sim_same_all_turns_prev <- df_simulated %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    n_valid = sum(!is.na(ai_valence_lag1)),
    data_source = "Simulated"
  )

cor_same_combined_all_turns_prev <- bind_rows(cor_field_same_all_turns_prev, cor_sim_same_all_turns_prev
  mutate(alignment_type = "Previous Turn All Turns")

cat("=== Previous-Turn All Turns Alignment Summary ===\n")
```

```
## === Previous-Turn All Turns Alignment Summary ===
```

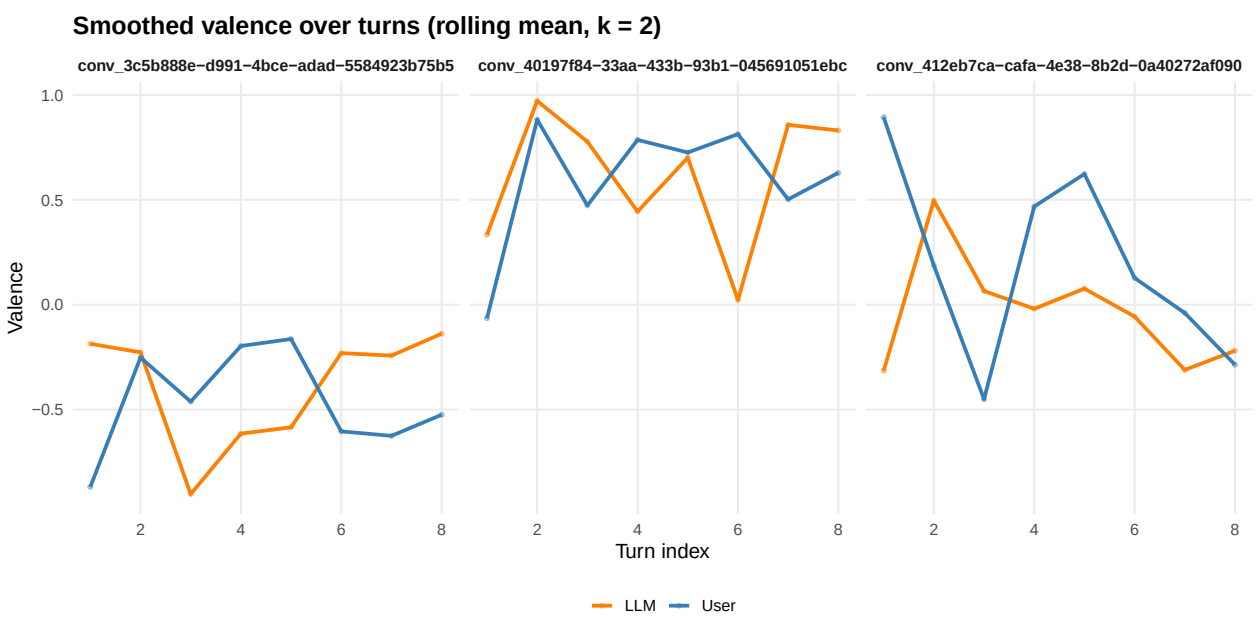
```r
cor_same_combined_all_turns_prev %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_total = sum(n_turns),
    n_valid = sum(n_valid)
  ) %>%
  print()
```

```
## # A tibble: 2 x 6
##   data_source mean_rho sd_rho median_rho n_total n_valid
##   <chr>          <dbl>  <dbl>      <dbl>   <int>   <int>
## 1 Field          0.365     NA      0.365     696     609
## 2 Simulated      0.425     NA      0.425      30      24
```

4.5 4.3.3 Probe Sentiment Narrative Development

```r
p_probe_sentiment <- df_field %>%
  mutate(time = ymd_hms(timestamp)) %>%

  group_by(conversation_id) %>%
  arrange(time, .by_group = TRUE) %>%
  mutate(turn_index = row_number()) %>%
  ungroup() %>%
  filter(conversation_id %in% unique(conversation_id)[22:24]) %>%
  pivot_longer(cols = c(user_valence, ai_valence),
```

```r
                 names_to = "source", values_to = "valence") %>%
mutate(
  source = recode(source, "user_valence" = "User", "ai_valence" = "LLM")
) %>%
group_by(conversation_id, source) %>%            # NEW: smooth separately per convo + source
arrange(turn_index, .by_group = TRUE) %>%        # NEW: make sure order is correct
mutate(
  valence_smoothed = rollmean(valence, k = 1,     # CHANGE: k = 3 (or 5 if you want very smooth)
                              fill = NA,
                              align = "center")
) %>%
ungroup() %>%
ggplot(aes(x = turn_index, y = valence_smoothed, color = source)) +
  geom_line(size = 1.2) +                                # thicker lines
  geom_point(aes(y = valence), size = 1.2, alpha = 0.5) +# optional: show raw points faintly
  scale_color_manual(values = c("User" = "#377EB8", "LLM" = "#FF7F00")) +
  facet_wrap(~ conversation_id, scales = "free_x") +
  labs(
    title = "Smoothed valence over turns (rolling mean, k = 2)",  # keep this in sync with k
    x = "Turn index",
    y = "Valence",
    color = NULL
  ) +
  theme_minimal(base_size = 14) +                        # nicer base size
  theme(
    legend.position = "bottom",                          # cleaner legend
    panel.grid.minor = element_blank(),
    strip.text = element_text(face = "bold"),
    plot.title = element_text(face = "bold", hjust = 0)
  )

print(p_probe_sentiment)
```



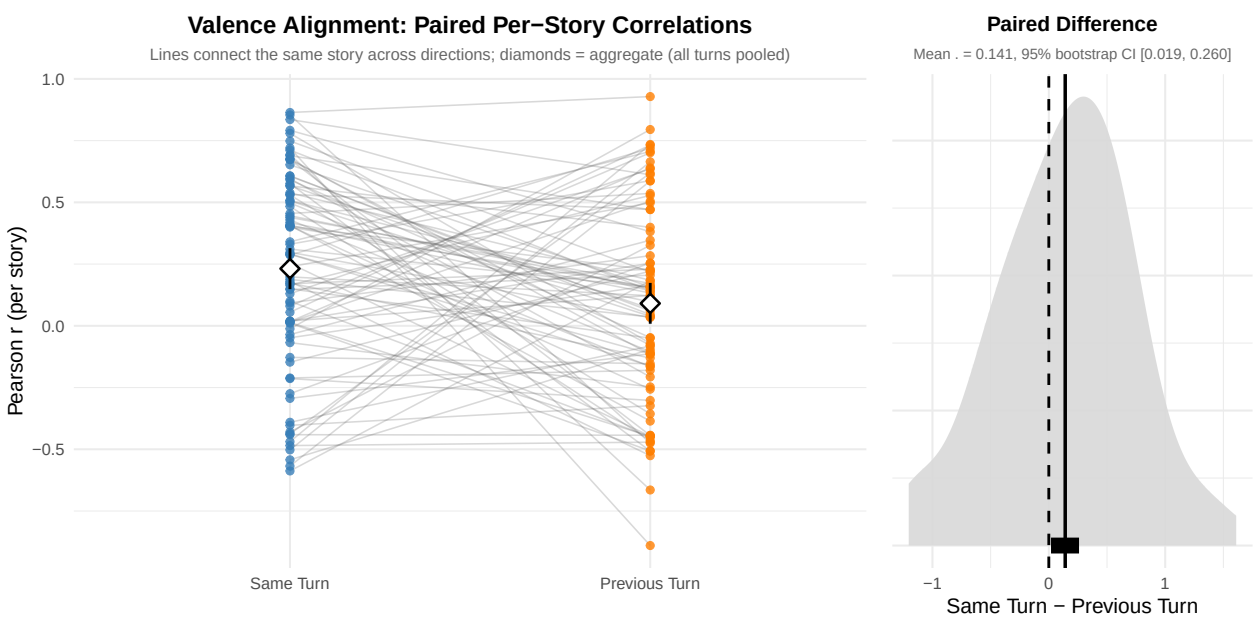
```r
ggsave(file.path(GRAPHS_DIR, "valence_probe_temporal.png"),
       p_probe_sentiment, width = 12, height = 6, dpi = 300)
```

4.6 4.4.1 Alignment Comparison Plot

```r
# Combine all correlations
cor_all <- bind_rows(cor_same_combined, cor_prev_combined) %>%
  mutate(
    alignment_type = factor(alignment_type,
                            levels = c("Same Turn", "Previous Turn"))
  )



# ---- Helper: bootstrap CI for the mean (percentile CI) ----
boot_mean_ci <- function(x, R = 5000, conf = 0.95) {
  x <- x[is.finite(x)]
  b <- boot::boot(
    data = x,
    statistic = function(d, i) mean(d[i], na.rm = TRUE),
    R = R
  )
  ci <- boot::boot.ci(b, type = "perc", conf = conf)$percent[4:5]
  list(mean = mean(x, na.rm = TRUE), lo = ci[1], hi = ci[2])
}

# ---- 1) Build paired wide table (one row per story) ----
pairs_field <- cor_all %>%
  filter(data_source == "Field") %>%
  select(conversation_id, alignment_type, rho) %>%
  mutate(alignment_type = as.character(alignment_type)) %>%
  pivot_wider(names_from = alignment_type, values_from = rho) %>%
  drop_na(`Same Turn`, `Previous Turn`) %>%
  mutate(diff_same_minus_prev = `Same Turn` - `Previous Turn`)

# ---- 2) Long format for connected dots ----
pairs_long <- pairs_field %>%
  select(conversation_id, `Same Turn`, `Previous Turn`) %>%
  pivot_longer(
    cols = c(`Same Turn`, `Previous Turn`),
    names_to = "alignment_type",
    values_to = "rho"
  ) %>%
  mutate(alignment_type = factor(alignment_type, levels = c("Same Turn", "Previous Turn")))

# ---- 3) Mean + CI per condition (bootstrap) ----
ci_same <- boot_mean_ci(pairs_long$rho[pairs_long$alignment_type == "Same Turn"])
ci_prev <- boot_mean_ci(pairs_long$rho[pairs_long$alignment_type == "Previous Turn"])

ci_by_type <- tibble::tibble(
  alignment_type = factor(c("Same Turn", "Previous Turn"), levels = c("Same Turn", "Previous Turn")),
  mean = c(ci_same$mean, ci_prev$mean),
  lo   = c(ci_same$lo,   ci_prev$lo),
```

```r
  hi   = c(ci_same$hi,   ci_prev$hi)
)

# ---- 4) Panel A: paired connected dots + aggregate diamonds ----
p_paired <- ggplot(pairs_long, aes(x = alignment_type, y = rho)) +
  geom_line(aes(group = conversation_id), alpha = 0.25, linewidth = 0.5, color = "grey40") +
  geom_point(aes(color = alignment_type), size = 2.2, alpha = 0.8) +
  # Mean + bootstrap CI overlay
geom_pointrange(
  data = ci_by_type,
  aes(x = alignment_type, y = mean, ymin = lo, ymax = hi),
  inherit.aes = FALSE,
  linewidth = 0.9
) +
geom_point(
  data = ci_by_type,
  aes(x = alignment_type, y = mean),
  inherit.aes = FALSE,
  size = 4.0,
  shape = 23,
  fill = "white",
  color = "black",
  stroke = 1.2
) +
  scale_color_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00")) +
  scale_fill_manual(values  = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00")) +
  labs(
    title = "Valence Alignment: Paired Per-Story Correlations",
    subtitle = "Lines connect the same story across directions; diamonds = aggregate (all turns pooled)
    x = NULL,
    y = "Pearson r (per story)"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
    plot.subtitle = element_text(hjust = 0.5, size = 11, color = "grey40")
  )

# ---- 5) Panel B: paired difference distribution + CI ----
ci_diff <- boot_mean_ci(pairs_field$diff_same_minus_prev)
ci_diff_df <- tibble::tibble(mean = ci_diff$mean, lo = ci_diff$lo, hi = ci_diff$hi)

p_diff <- ggplot(pairs_field, aes(x = diff_same_minus_prev)) +
  geom_density(fill = "grey85", color = NA, alpha = 0.9) +
  geom_vline(xintercept = 0, linetype = "dashed", linewidth = 0.9) +
  # CI bar at y=0 (simple, readable)
  geom_segment(
    data = ci_diff_df,
    aes(x = lo, xend = hi, y = 0, yend = 0),
    inherit.aes = FALSE,
    linewidth = 5
  ) +
```

```r
  geom_vline(data = ci_diff_df, aes(xintercept = mean), linewidth = 1.1) +
  labs(
    title = "Paired Difference",
    subtitle = sprintf("Mean Δ = %.3f, 95%% bootstrap CI [%.3f, %.3f]",
                       ci_diff_df$mean, ci_diff_df$lo, ci_diff_df$hi),
    x = "Same Turn - Previous Turn",
    y = NULL
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    plot.subtitle = element_text(hjust = 0.5, size = 10, color = "grey40")
  )

# ---- 6) Combine panels ----
p_combined <- p_paired + p_diff + plot_layout(widths = c(2.2, 1))

print(p_combined)
```

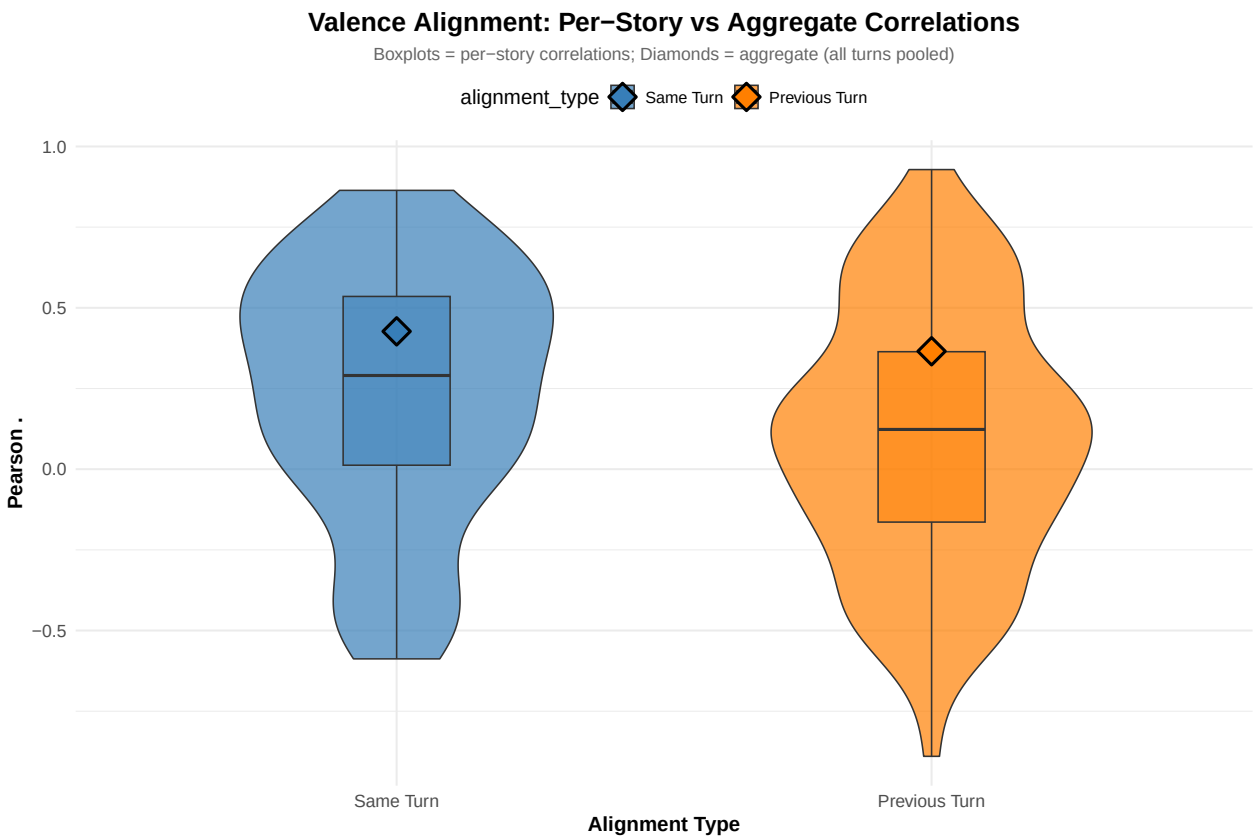



```r
# ---- 7) Save (match your GRAPHS_DIR style) ----
ggsave(file.path(GRAPHS_DIR, "valence_alignment_paired_difference.png"), p_combined, width = 13, height
```

4.7 4.4.2 Comparison: Per-Story vs Aggregate Correlations

```r
# Prepare aggregate correlation data for overlay
agg_same <- cor_same_combined_all_turns %>%
  mutate(alignment_type = "Same Turn")

agg_prev <- cor_same_combined_all_turns_prev %>%
  mutate(alignment_type = "Previous Turn") %>%
  select(data_source, rho, alignment_type)
```

```r
agg_data <- bind_rows(agg_same, agg_prev) %>%
  filter(data_source == "Field") %>%
  mutate(
    alignment_type = factor(alignment_type,
                            levels = c("Same Turn", "Previous Turn"))
  )

# Create plot with per-story boxplots and aggregate overlay
p_comparison <- cor_all %>%
  filter(data_source == "Field") %>%
  ggplot(aes(x = alignment_type, y = rho, fill = alignment_type)) +
  # Per-story boxplots
  geom_violin(width = 0.6, alpha = 0.7) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.5,
               position = position_dodge(width = 0.75)) +
  # Aggregate correlations as large points
  geom_point(data = agg_data,
             aes(x = alignment_type, y = rho, fill = alignment_type),
             position = position_dodge(width = 0.75),
             size = 6, shape = 23, color = "black", stroke = 1.5) +
  scale_fill_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00")) +
  scale_color_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00", name = "Alignment
  labs(
    title = "Valence Alignment: Per-Story vs Aggregate Correlations",
    subtitle = "Boxplots = per-story correlations; Diamonds = aggregate (all turns pooled)",
    x = "Alignment Type",
    y = "Pearson  "
  ) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40"),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

print(p_comparison)
```




```r
ggsave(file.path(GRAPHS_DIR, "valence_alignment_comparison_with_aggregate.png"),
       p_comparison, width = 12, height = 8, dpi = 300)

# Print summary comparison
cat("\n=== Per-Story vs Aggregate Correlation Comparison ===\n")
```

```
##
## === Per-Story vs Aggregate Correlation Comparison ===
```

```r
comparison_summary <- bind_rows(
  cor_all %>%
    group_by(data_source, alignment_type) %>%
    summarise(
      mean_rho = mean(rho, na.rm = TRUE),
      method = "Per-Story Mean",
      .groups = "drop"
    ),
  agg_data %>%
    mutate(mean_rho = rho, method = "Aggregate All Turns") %>%
    select(data_source, alignment_type, mean_rho, method)
) %>%
  pivot_wider(names_from = method, values_from = mean_rho)

print(comparison_summary)
```

```
## # A tibble: 4 x 4
##   data_source alignment_type `Per-Story Mean` `Aggregate All Turns`
##   <chr>       <fct>                     <dbl>                 <dbl>
```

```
## 1 Field       Same Turn                0.232                  0.427
## 2 Field       Previous Turn            0.0908                 0.365
## 3 Simulated   Same Turn               -0.246                 NA
## 4 Simulated   Previous Turn            0.533                 NA
```

```r
# We assume df_field already has:


# 1) Build long-format dataset with both directions in a common structure

# Same-turn: User -> AI
df_same_dir <- df_field %>%
  transmute(
    conversation_id,
    turn,
    alignment_direction = "User_to_AI",
    predictor_valence = user_valence,
    response_valence = ai_valence
  )

# Previous-turn: AI -> User (response to previous AI valence)
df_prev_dir <- df_field %>%
  filter(!is.na(ai_valence_lag1)) %>%   # ensure valid lag values
  transmute(
    conversation_id,
    turn,
    alignment_direction = "AI_to_User",
    predictor_valence = ai_valence_lag1,
    response_valence = user_valence
  )

# Combine into one long dataset
df_alignment_long <- bind_rows(df_same_dir, df_prev_dir) %>%
  mutate(
    alignment_direction = factor(
      alignment_direction,
      levels = c("User_to_AI", "AI_to_User")
    )
  )

cat("=== Directional Alignment Long-Format Data ===\n")
```

```
## === Directional Alignment Long-Format Data ===
```

```r
df_alignment_long %>%
  group_by(alignment_direction) %>%
  summarise(
    n_rows = n(),
    mean_predictor = mean(predictor_valence, na.rm = TRUE),
    mean_response  = mean(response_valence,  na.rm = TRUE),
    n_conversations = n_distinct(conversation_id),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   alignment_direction n_rows mean_predictor mean_response n_conversations
##   <fct>                <int>          <dbl>         <dbl>           <int>
## 1 User_to_AI             696       -0.0891        0.00381              87
## 2 AI_to_User             609       -0.00336      -0.100                87
```

```
# standardize within the direction groups to ensure beta = correlation
df_alignment_std <- df_alignment_long %>%
  group_by(conversation_id, alignment_direction) %>%
  mutate(
    predictor_z = as.numeric(scale(predictor_valence)),
    response_z = as.numeric(scale(response_valence))
  ) %>%
  ungroup()



model_direction <- lmer(
  response_z ~ predictor_z * alignment_direction + (1 | conversation_id),
  data = df_alignment_std
)
cat("=== Mixed Effects Model (Standardized): Directional Alignment (User -> AI vs AI -> User) ===\n")
```

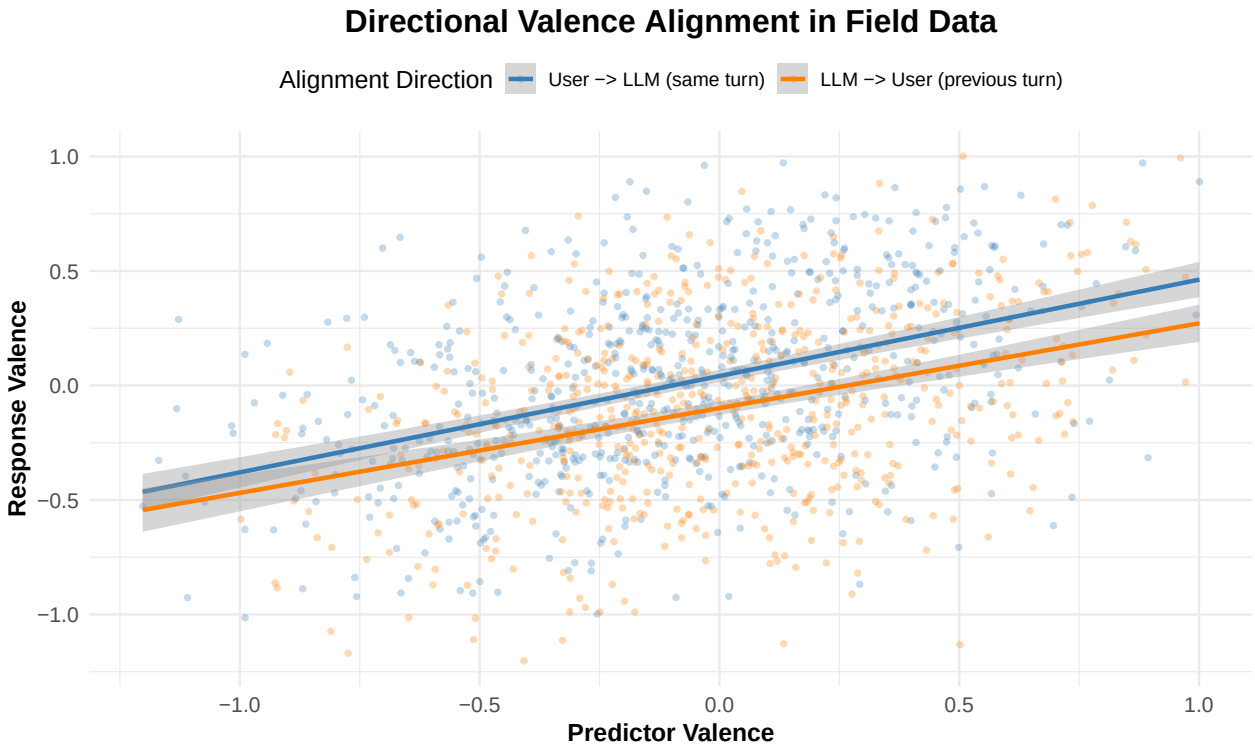
```
## === Mixed Effects Model (Standardized): Directional Alignment (User -> AI vs AI -> User) ===
```

```
## === Mixed Effects Model: Directional Alignment (User -> AI vs AI -> User) ===
direction_summary <- summary(model_direction)
print(round(direction_summary$coefficients, 4))
```

```
##                                          Estimate Std. Error   df t value
## (Intercept)                                0.0000     0.0348 1301  0.0000
## predictor_z                                0.2316     0.0372 1301  6.2322
## alignment_directionAI_to_User              0.0000     0.0509 1301  0.0000
## predictor_z:alignment_directionAI_to_User -0.1408     0.0547 1301 -2.5740
##                                          Pr(>|t|)
## (Intercept)                                1.0000
## predictor_z                                0.0000
## alignment_directionAI_to_User              1.0000
## predictor_z:alignment_directionAI_to_User  0.0102
```

```
# 3) Visualise directional alignment with separate regression lines

p_direction_alignment <- df_alignment_long %>%
  ggplot(aes(
    x = predictor_valence,
    y = response_valence,
    color = alignment_direction
  )) +
  geom_point(alpha = 0.3, size = 1.3) +
  geom_smooth(
    method = "lm",
    se = TRUE,
    fullrange = TRUE,
    size = 1.2
  ) +
  scale_color_manual(
```

```r
    values = c("User_to_AI" = "#377EB8", "AI_to_User" = "#FF7F00"),
    name = "Alignment Direction",
    labels = c("User → LLM (same turn)", "LLM → User (previous turn)")
  ) +
  labs(
    title = "Directional Valence Alignment in Field Data",
    x = "Predictor Valence",
    y = "Response Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

print(p_direction_alignment)
```




```r
ggsave(
  file.path(GRAPHS_DIR, "directional_valence_alignment_field.png"),
  p_direction_alignment,
  width = 10,
  height = 6,
  dpi = 300
)
```

4.8 4.6 Statistical Tests: Alignment Differences

```r
# Test 1: Same-turn alignment difference
t_test_same <- t.test(rho ~ data_source, data = cor_same_combined)
#cat("=== T-test: Same-Turn Alignment ===\n")
#print(t_test_same)

# Test 2: Previous-turn alignment difference
t_test_prev <- t.test(rho ~ data_source, data = cor_prev_combined)
#cat("\n=== T-test: Previous-Turn Alignment ===\n")
#print(t_test_prev)

# Test 3: One-sample t-tests against zero (is alignment significant?)
cat("\n=== One-Sample Tests Against Zero ===\n")
```

```
##
## === One-Sample Tests Against Zero ===
```

```r
cor_all %>%
  group_by(data_source, alignment_type) %>%
  summarise(
    t_stat = t.test(rho, mu = 0)$statistic,
    p_value = t.test(rho, mu = 0)$p.value,
    mean_rho = mean(rho, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 4 x 6
##   data_source alignment_type t_stat     p_value mean_rho     n
##   <chr>       <fct>           <dbl>       <dbl>    <dbl> <int>
## 1 Field       Same Turn        5.55 0.000000312   0.232     87
## 2 Field       Previous Turn    2.14 0.0349        0.0908    87
## 3 Simulated   Same Turn       -1.37 0.229        -0.246      6
## 4 Simulated   Previous Turn    5.84 0.00209       0.533      6
```

```r
# Test 4: Paired t-test (Directional Asymmetry within Story)
# We join same and prev correlations by conversation_id to create pairs
cat("\n=== Paired T-Test: Directional Asymmetry (Same vs Prev) ===\n")
```

```
##
## === Paired T-Test: Directional Asymmetry (Same vs Prev) ===
```

```r
# Prepare Field pairs
pairs_field <- inner_join(
  cor_field_same %>% select(conversation_id, rho_same = rho),
  cor_field_prev %>% select(conversation_id, rho_prev = rho),
  by = "conversation_id"
)

# Apply Fisher z-transformation before t-test
t_test_paired_field <- t.test(atanh(pairs_field$rho_same), atanh(pairs_field$rho_prev), paired = TRUE)
cat("Field Data: Paired t-test (Fisher z-transformed Same Turn - Previous Turn)\n")
```

```
## Field Data: Paired t-test (Fisher z-transformed Same Turn - Previous Turn)
```

```r
print(t_test_paired_field)
```

```
##
##  Paired t-test
##
## data:  atanh(pairs_field$rho_same) and atanh(pairs_field$rho_prev)
## t = 2.2023, df = 86, p-value = 0.03032
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  0.01613391 0.31536192
## sample estimates:
## mean difference
##       0.1657479
```

```r
# Calculate back-transformed means for reporting
mean_z_same <- mean(atanh(pairs_field$rho_same), na.rm = TRUE)
mean_z_prev <- mean(atanh(pairs_field$rho_prev), na.rm = TRUE)

cat("\n--- Interpretation (Back-transformed means) ---\n")
```

```
##
## --- Interpretation (Back-transformed means) ---
```

```r
cat("Mean Same-Turn r (User->AI):", round(tanh(mean_z_same), 3), "\n")
```

```
## Mean Same-Turn r (User->AI): 0.272
```

```r
cat("Mean Prev-Turn r (AI->User):", round(tanh(mean_z_prev), 3), "\n")
```

```
## Mean Prev-Turn r (AI->User): 0.112
```

```r
# Prepare Simulated pairs
pairs_sim <- inner_join(
  cor_sim_same %>% select(conversation_id, rho_same = rho),
  cor_sim_prev %>% select(conversation_id, rho_prev = rho),
  by = "conversation_id"
)

# Apply Fisher z-transformation before t-test
t_test_paired_sim <- t.test(atanh(pairs_sim$rho_same), atanh(pairs_sim$rho_prev), paired = TRUE)
cat("\nSimulated Data: Paired t-test (Fisher z-transformed Same Turn - Previous Turn)\n")
```

```
##
## Simulated Data: Paired t-test (Fisher z-transformed Same Turn - Previous Turn)
```

```r
print(t_test_paired_sim)
```

```
##
##  Paired t-test
##
## data:  atanh(pairs_sim$rho_same) and atanh(pairs_sim$rho_prev)
## t = -3.4821, df = 5, p-value = 0.01762
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -1.6343524 -0.2461251
## sample estimates:
## mean difference
##      -0.9402387
```

```r
# Save results
alignment_test_results <- tibble(
  test = c("Same-turn: Field vs Simulated", "Previous-turn: Field vs Simulated"),
  t_statistic = c(t_test_same$statistic, t_test_prev$statistic),
  p_value = c(t_test_same$p.value, t_test_prev$p.value),
  mean_field = c(mean(cor_field_same$rho, na.rm = TRUE),
                 mean(cor_field_prev$rho, na.rm = TRUE)),
  mean_sim = c(mean(cor_sim_same$rho, na.rm = TRUE),
               mean(cor_sim_prev$rho, na.rm = TRUE))
)

write_csv(alignment_test_results,
          file.path(TABLES_DIR, "alignment_statistical_tests.csv"))
```

4.9 Cosine Distribution Difference Test

```r
df_field_cosine_long <- df_field %>%
  pivot_longer(cols = c(cosine_sim_same, cosine_sim_prev),
               names_to = "direction", values_to = "cosine_sim") %>%
  mutate(direction = recode(direction,
                            cosine_sim_same = "Same Turn",
                            cosine_sim_prev = "Previous Turn"),
         direction = factor(direction, levels = c("Same Turn", "Previous Turn"))
         )
df_field_cosine_long_prev_self <- df_field %>%
  pivot_longer(cols = c(cosine_sim_user_self, cosine_sim_prev),
               names_to = "direction", values_to = "cosine_sim") %>%
  mutate(direction = recode(direction,
                            cosine_sim_user_self = "Self",
                            cosine_sim_prev = "Interlocutor"),
         direction = factor(direction, levels = c("Self", "Interlocutor"))
         )
df_field_cosine_long_prev_self_ai <- df_field %>%
  pivot_longer(cols = c(cosine_sim_ai_self, cosine_sim_same),
               names_to = "direction", values_to = "cosine_sim") %>%
  mutate(direction = recode(direction,
                            cosine_sim_ai_self = "Self",
                            cosine_sim_same = "Interlocutor"),
         direction = factor(direction, levels = c("Self", "Interlocutor"))
         )


# Calculate means for vertical lines
means_field <- df_field_cosine_long %>%
  group_by(direction) %>%
  summarise(mean_val = mean(cosine_sim, na.rm = TRUE), .groups = "drop")




cosine_difference_mixed_model_self_user <- lmer(cosine_sim ~ direction + (direction | conversation_id),
cat("\n=== Cosine Sim: User (Self vs Interlocutor) ===\n")
```

```
##
## === Cosine Sim: User (Self vs Interlocutor) ===
```

```r
print(round(summary(cosine_difference_mixed_model_self_user)$coefficients, 4))
```

```
##                       Estimate Std. Error      df  t value Pr(>|t|)
## (Intercept)             0.6897     0.0041 86.0000 169.6669   0.0000
## directionInterlocutor  -0.0117     0.0040 85.9997  -2.9192   0.0045
```

```r
cosine_difference_mixed_model_self_ai <- lmer(cosine_sim ~ direction + (direction | conversation_id), da
cat("\n=== Cosine Sim: AI (Self vs Interlocutor) ===\n")
```

```
##
## === Cosine Sim: AI (Self vs Interlocutor) ===
```

```r
print(round(summary(cosine_difference_mixed_model_self_ai)$coefficients, 4))
```

```
##                       Estimate Std. Error      df  t value Pr(>|t|)
## (Intercept)             0.6860     0.0041 86.0120 167.2791   0.0000
## directionInterlocutor   0.0054     0.0036 86.2026   1.5244   0.1311
```

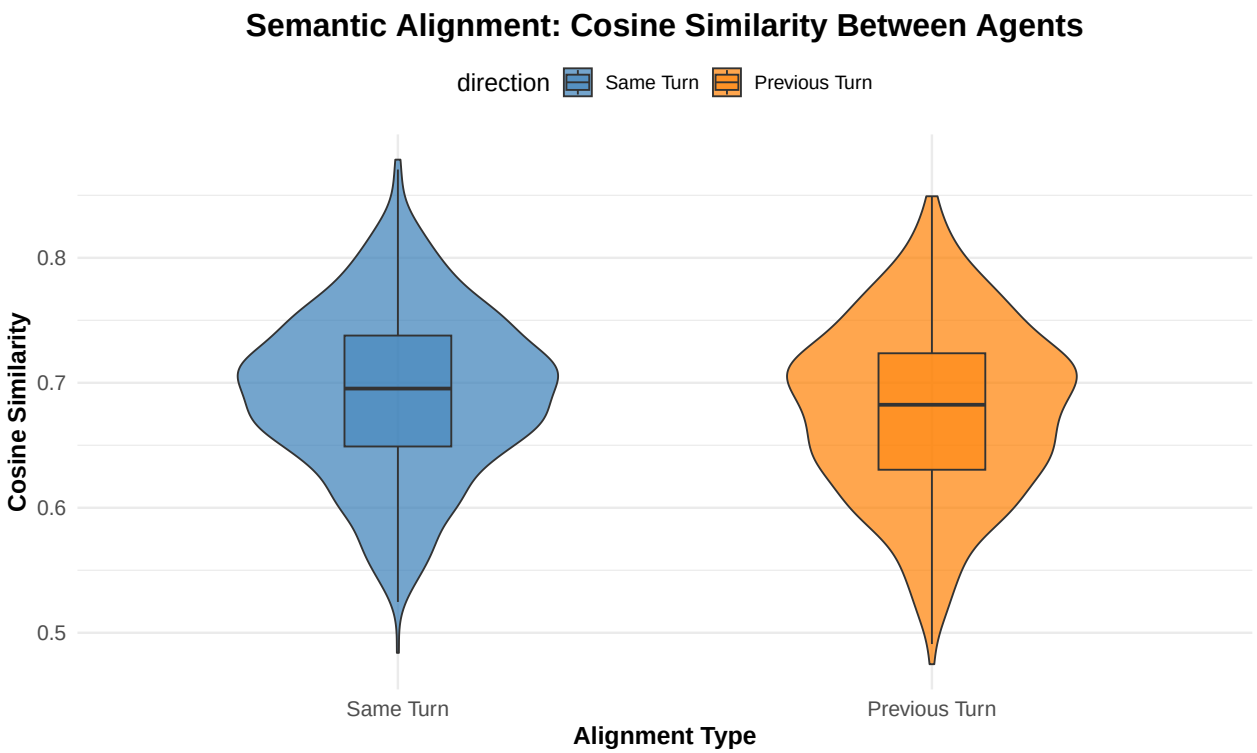
```r
cosine_difference_mixed_model <- lmer(cosine_sim ~ direction + (direction | conversation_id), data=df_fi
cat("\n=== Cosine Sim: General Direction Comparison ===\n")
```

```
##
## === Cosine Sim: General Direction Comparison ===
```

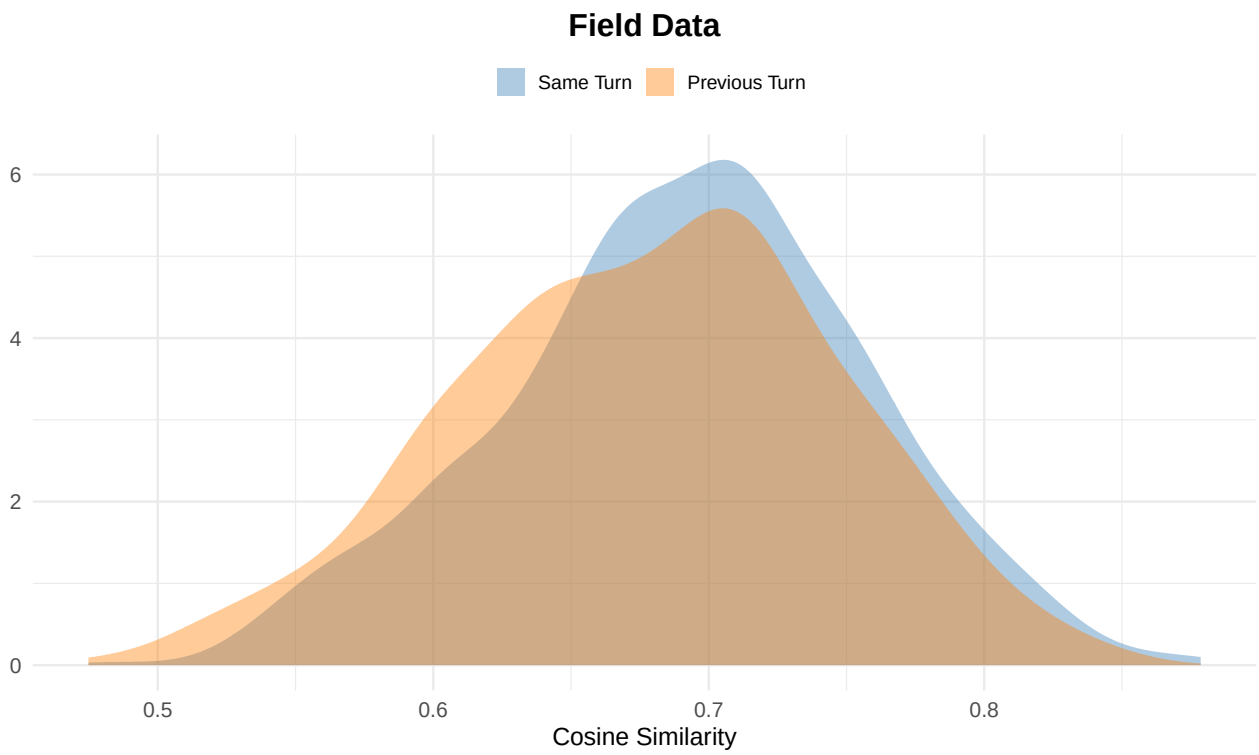
```r
print(round(summary(cosine_difference_mixed_model)$coefficients, 4))
```

```
##                        Estimate Std. Error       df  t value Pr(>|t|)
## (Intercept)              0.6914     0.0036  94.1491 192.3986    0e+00
## directionPrevious Turn  -0.0134     0.0034 691.2310  -3.9250    1e-04
```

```r
# Create plot with per-story boxplots and aggregate overlay
p_comparison_cosine_violin <- df_field_cosine_long %>%
  ggplot(aes(x = direction, y = cosine_sim, fill = direction)) +
  # Per-story boxplots
  geom_violin(width = 0.6, alpha = 0.7) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.5,
               position = position_dodge(width = 0.75)) +
  scale_fill_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00")) +
  scale_color_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00", name = "direction
  labs(
    title = "Semantic Alignment: Cosine Similarity Between Agents",
    x = "Alignment Type",
    y = "Cosine Similarity"
  ) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40"),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

print(p_comparison_cosine_violin)
```




```r
ggsave(file.path(GRAPHS_DIR, "cosine_sim_violin.png"),
       p_comparison_cosine_violin, width = 12, height = 8, dpi = 300)

p_field <- ggplot(df_field_cosine_long, aes(x = cosine_sim, fill = direction)) +
  geom_density(alpha = 0.4, color = NA) +
  scale_fill_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00"), name = "") +
  scale_color_manual(values = c("Same Turn" = "#377EB8", "Previous Turn" = "#FF7F00")) +
  labs(title = "Field Data", x = "Cosine Similarity", y = NULL) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.text = element_text(size = 12)
  )
print(p_field)
```



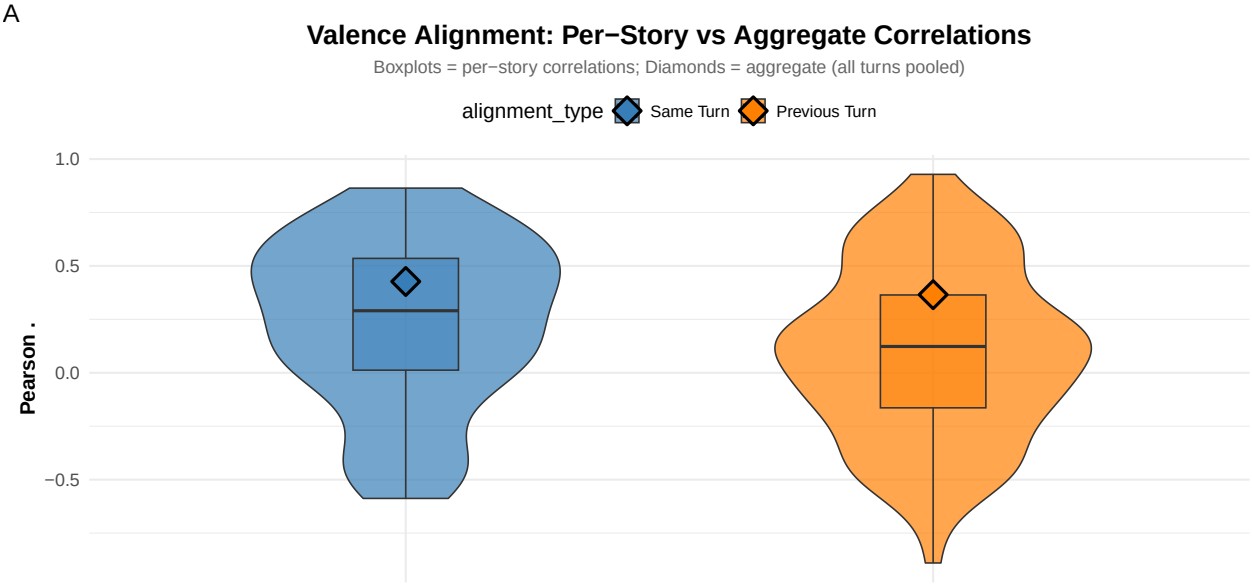

```
ggsave(
  file.path(GRAPHS_DIR, "directional_cosine_alignment_field.png"),
  p_field,
  width = 10,
  height = 6,
  dpi = 300
)
```

4.10 Combined Alignment Plot

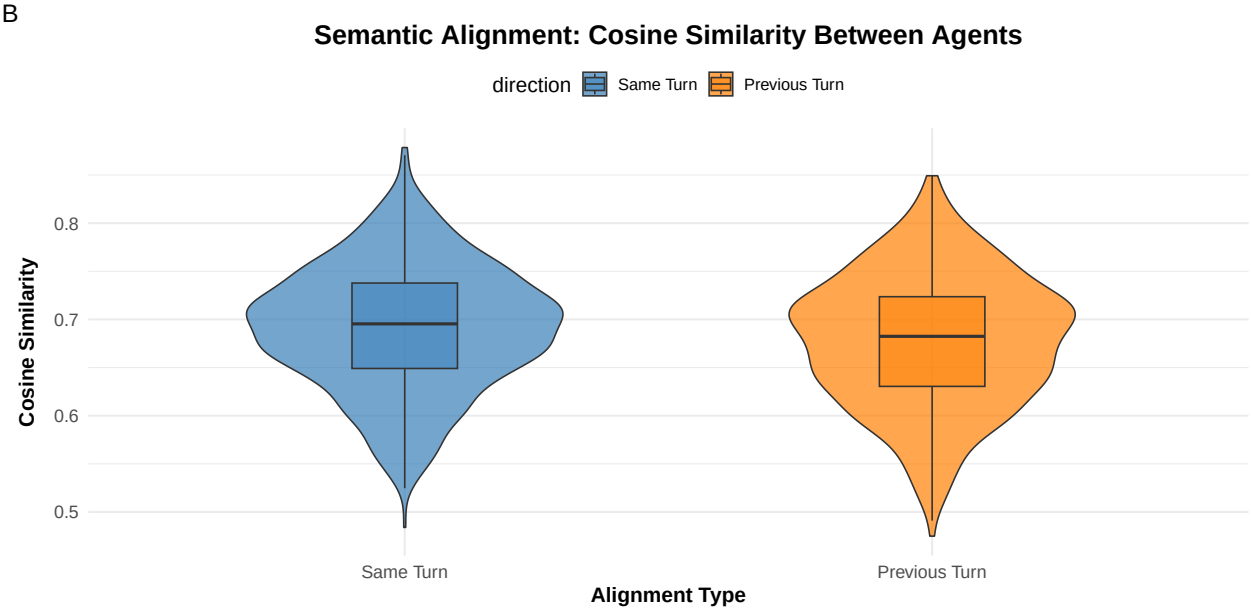
```
# Combine Valence Alignment (p_comparison) and Semantic Alignment (p_comparison_cosine_violin)
# Note: p_comparison is defined in chunk 'alignment-comparison-aggregate'
#       p_comparison_cosine_violin is defined in the chunk above

# Remove x-axis title and text from the top plot to avoid redundancy
p_comparison_clean <- p_comparison +
  theme(axis.title.x = element_blank(),
        axis.text.x = element_blank(),
        axis.ticks.x = element_blank())

combined_alignment_plot <- (p_comparison_clean / p_comparison_cosine_violin) +
  plot_annotation(
    title = "Comparison of Valence and Semantic Alignment",
    tag_levels = 'A'
  )

print(combined_alignment_plot)
```

Comparison of Valence and Semantic Alignment

A



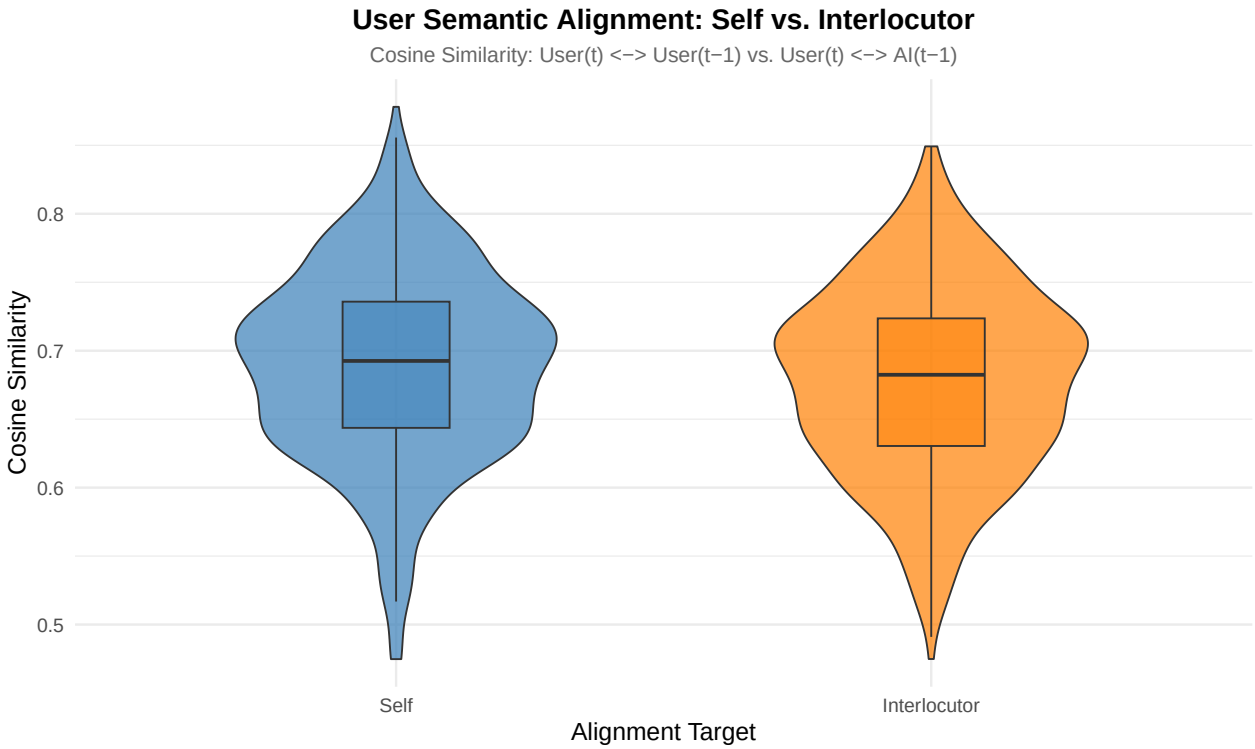
B




```
ggsave(file.path(GRAPHS_DIR, "combined_valence_semantic_alignment.png"),
       combined_alignment_plot, width = 12, height = 12, dpi = 300)
```

```
# Plot for User: Self vs Interlocutor Cosine Similarity
p_cosine_user_self_vs_interlocutor <- df_field_cosine_long_prev_self %>%
  ggplot(aes(x = direction, y = cosine_sim, fill = direction)) +
  geom_violin(width = 0.6, alpha = 0.7) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.5,
               position = position_dodge(width = 0.75)) +
  scale_fill_manual(values = c("Self" = "#377EB8", "Interlocutor" = "#FF7F00")) +
  labs(
    title = "User Semantic Alignment: Self vs. Interlocutor",
    subtitle = "Cosine Similarity: User(t)   User(t-1) vs. User(t)   AI(t-1)",
    x = "Alignment Target",
```

```
    y = "Cosine Similarity"
  ) +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
    plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40")
  )

print(p_cosine_user_self_vs_interlocutor)
```




```
ggsave(file.path(GRAPHS_DIR, "cosine_similarity_user_self_vs_interlocutor.png"),
       p_cosine_user_self_vs_interlocutor, width = 10, height = 6, dpi = 300)
```

4.11 Valence Distance Difference Test

```
# Calculate valence distances
df_field_val <- df_field %>%
  mutate(
    val_dist_same = abs(user_valence - ai_valence),
    val_dist_prev = abs(user_valence - ai_valence_lag1),
    val_dist_user_self = abs(user_valence - user_valence_lag1),
    val_dist_ai_self = abs(ai_valence - ai_valence_lag1)
  )

# User: Self vs Partner (Prev)
df_field_val_long_user <- df_field_val %>%
  pivot_longer(cols = c(val_dist_user_self, val_dist_prev),
               names_to = "direction", values_to = "val_dist") %>%
  mutate(direction = recode(direction,
```

```r
                              val_dist_user_self = "Self",
                              val_dist_prev = "Interlocutor"),
         direction = factor(direction, levels = c("Self", "Interlocutor")))

# AI: Self vs Partner (Same)
df_field_val_long_ai <- df_field_val %>%
  pivot_longer(cols = c(val_dist_ai_self, val_dist_same),
               names_to = "direction", values_to = "val_dist") %>%
  mutate(direction = recode(direction,
                              val_dist_ai_self = "Self",
                              val_dist_same = "Interlocutor"),
         direction = factor(direction, levels = c("Self", "Interlocutor")))

cat("\n=== Valence Distance Model: User (Self vs Interlocutor) ===\n")
```

```
##
## === Valence Distance Model: User (Self vs Interlocutor) ===
```

```r
val_model_user <- lmer(val_dist ~ direction + (direction | conversation_id),
                       data = df_field_val_long_user)
print(round(summary(val_model_user)$coefficients, 4))
```

```
##                       Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)             0.3802     0.0145  94.256 26.2178   0.0000
## directionInterlocutor  -0.0126     0.0155 686.747 -0.8129   0.4166
```

```r
cat("\n=== Valence Distance Model: AI (Self vs Interlocutor) ===\n")
```

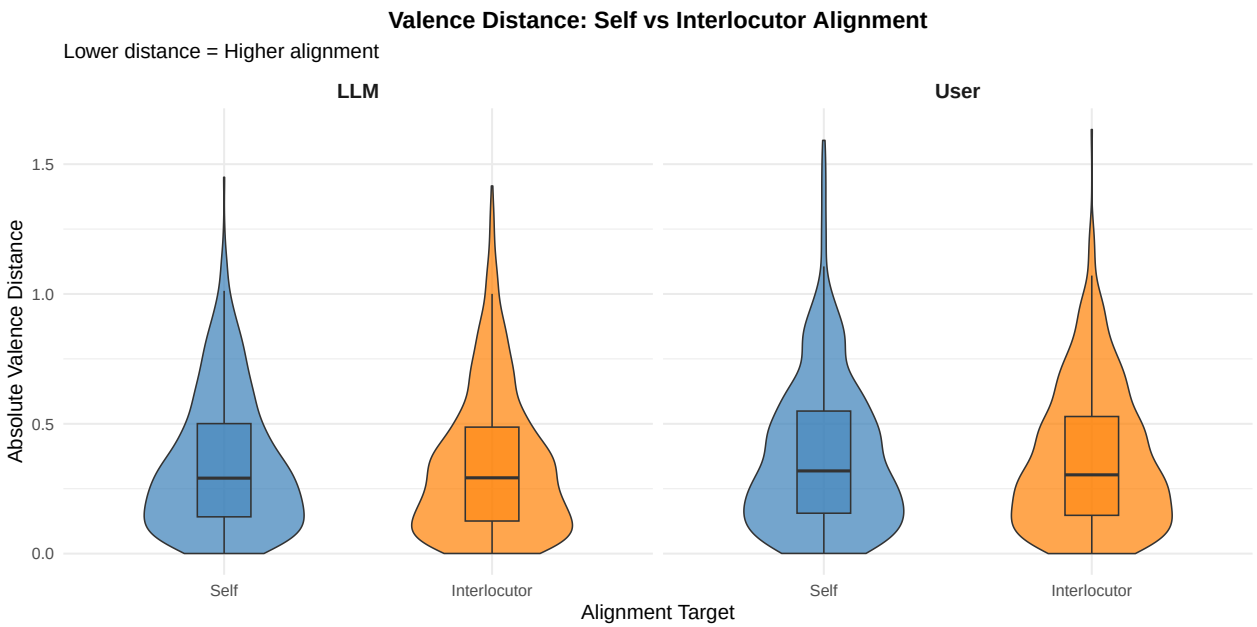
```
##
## === Valence Distance Model: AI (Self vs Interlocutor) ===
```

```r
val_model_ai <- lmer(val_dist ~ direction + (direction | conversation_id),
                     data = df_field_val_long_ai)
print(round(summary(val_model_ai)$coefficients, 4))
```

```
##                       Estimate Std. Error        df t value Pr(>|t|)
## (Intercept)             0.3478     0.0106 1151.3136 32.7421   0.0000
## directionInterlocutor  -0.0056     0.0166   61.3057 -0.3381   0.7365
```

```r
# Combine for plotting
df_val_plot <- bind_rows(
  df_field_val_long_user %>% mutate(agent = "User"),
  df_field_val_long_ai %>% mutate(agent = "LLM")
)

p_val_dist <- df_val_plot %>%
  ggplot(aes(x = direction, y = val_dist, fill = direction)) +
  geom_violin(width = 0.6, alpha = 0.7) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.5,
               position = position_dodge(width = 0.75)) +
  facet_wrap(~ agent) +
  scale_fill_manual(values = c("Self" = "#377EB8", "Interlocutor" = "#FF7F00")) +
  labs(
    title = "Valence Distance: Self vs Interlocutor Alignment",
    subtitle = "Lower distance = Higher alignment",
    x = "Alignment Target",
    y = "Absolute Valence Distance"
```

```r
  ) +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
    strip.text = element_text(size = 14, face = "bold")
  )

print(p_val_dist)
```




```r
ggsave(file.path(GRAPHS_DIR, "valence_distance_comparison.png"),
       p_val_dist, width = 12, height = 6, dpi = 300)
```

4.12 4.7 Alignment Duration and Stability

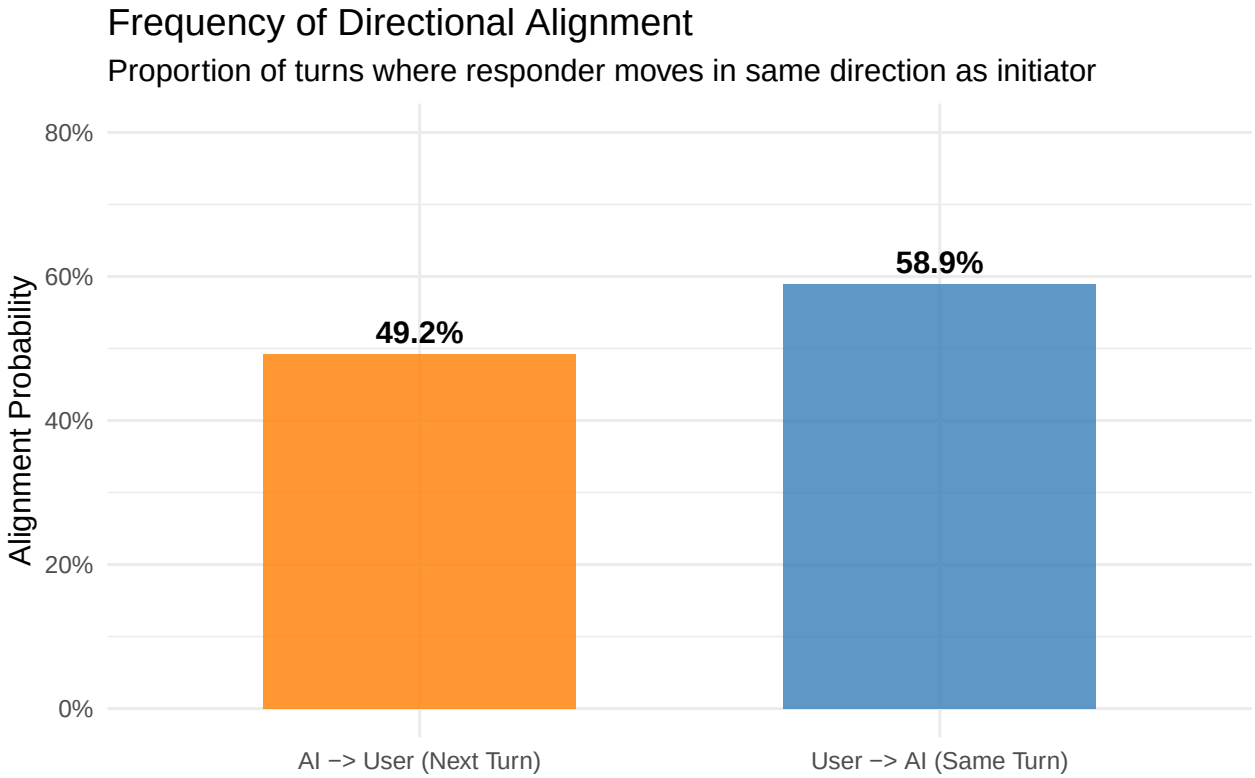
```r
# 1. Calculate Velocity and Alignment Flags
df_dynamics <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn) %>%
  mutate(
    # Calculate change from previous turn (Velocity)
    delta_user = user_valence - lag(user_valence),
    delta_ai   = ai_valence - lag(ai_valence),

    # Define Alignment: Signs of change match
    # Exclude cases where change is exactly 0 (no movement)

    # DIRECTION 1: User -> AI (Same Turn Response)
    # Does AI move in same direction User just moved?
    is_aligned_same = sign(delta_user) == sign(delta_ai) & delta_user != 0 & delta_ai != 0,

    # DIRECTION 2: AI -> User (Next Turn Response)
    # Does User move in same direction AI moved in previous turn?
    # Compare User(t) delta with AI(t-1) delta
```

```r
    # lag(delta_ai) is delta at t-1 (Change from t-2 to t-1)
    is_aligned_prev = sign(delta_user) == sign(lag(delta_ai)) & delta_user != 0 & lag(delta_ai) != 0
  ) %>%
  ungroup()

# 2. Extract Run Lengths (Streak Durations)
get_run_lengths <- function(x) {
  # Ensure input is a standard atomic vector (removes attributes)
  x <- as.vector(x)

  # Return empty if null or all NA
  if(length(x) == 0 || all(is.na(x))) return(numeric(0))

  # Filter NAs explicitly (cleaner than na.omit for rle)
  x <- x[!is.na(x)]

  # Double check length after filtering
  if(length(x) == 0) return(numeric(0))

  r <- rle(x)
  # Return lengths where value is TRUE
  return(r$lengths[r$values == TRUE])
}

alignment_durations <- df_dynamics %>%
  group_by(conversation_id) %>%
  summarise(
    duration_same = list(get_run_lengths(is_aligned_same)),
    duration_prev = list(get_run_lengths(is_aligned_prev))
  ) %>%
  pivot_longer(cols = c(duration_same, duration_prev),
               names_to = "direction",
               values_to = "duration") %>%
  unnest(duration) %>%
  mutate(
    direction_label = recode(direction,
                             duration_same = "User -> AI (Same Turn)",
                             duration_prev = "AI -> User (Next Turn)")
  )

# 3. Calculate General Alignment Frequency (Hypothesis: User->AI is more frequent)
alignment_frequency <- df_dynamics %>%
  pivot_longer(cols = c(is_aligned_same, is_aligned_prev),
               names_to = "direction",
               values_to = "is_aligned") %>%
  filter(!is.na(is_aligned)) %>%
  group_by(direction) %>%
  summarise(
    total_turns = n(),
    aligned_turns = sum(is_aligned),
    prop_aligned = aligned_turns / total_turns
  ) %>%
  mutate(
```

```
    direction_label = recode(direction,
                             is_aligned_same = "User -> AI (Same Turn)",
                             is_aligned_prev = "AI -> User (Next Turn)")
  )
```

4.12.1 4.7.1 Visualization: Alignment Frequency (Stability)

```
# Plot Overall Frequency
p_freq <- ggplot(alignment_frequency, aes(x = direction_label, y = prop_aligned, fill = direction_label
  geom_col(width = 0.6, alpha = 0.8) +
  geom_text(aes(label = scales::percent(prop_aligned, accuracy = 0.1)),
            vjust = -0.5, size = 5, fontface = "bold") +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 0.8)) + # Scale approx
  scale_fill_manual(values = c("User -> AI (Same Turn)" = "#377EB8",
                               "AI -> User (Next Turn)" = "#FF7F00")) +
  labs(
    title = "Frequency of Directional Alignment",
    subtitle = "Proportion of turns where responder moves in same direction as initiator",
    x = NULL,
    y = "Alignment Probability"
  ) +
  theme(legend.position = "none")

print(p_freq)
```




```
ggsave(file.path(GRAPHS_DIR, "alignment_frequency_bar.png"), p_freq, width = 8, height = 5)

# Statistical Test (Proportions Test)
prop_test <- prop.test(
```

```r
  x = alignment_frequency$aligned_turns,
  n = alignment_frequency$total_turns
)
cat("\n=== Proportions Test: Alignment Frequency ===\n")
```

```
##
## === Proportions Test: Alignment Frequency ===
```

```r
print(prop_test)
```

```
##
##  2-sample test for equality of proportions with continuity correction
##
## data:  alignment_frequency$aligned_turns out of alignment_frequency$total_turns
## X-squared = 10.309, df = 1, p-value = 0.001324
## alternative hypothesis: two.sided
## 95 percent confidence interval:
##  -0.15694805 -0.03735956
## sample estimates:
##    prop 1    prop 2
## 0.4923372 0.5894910
```

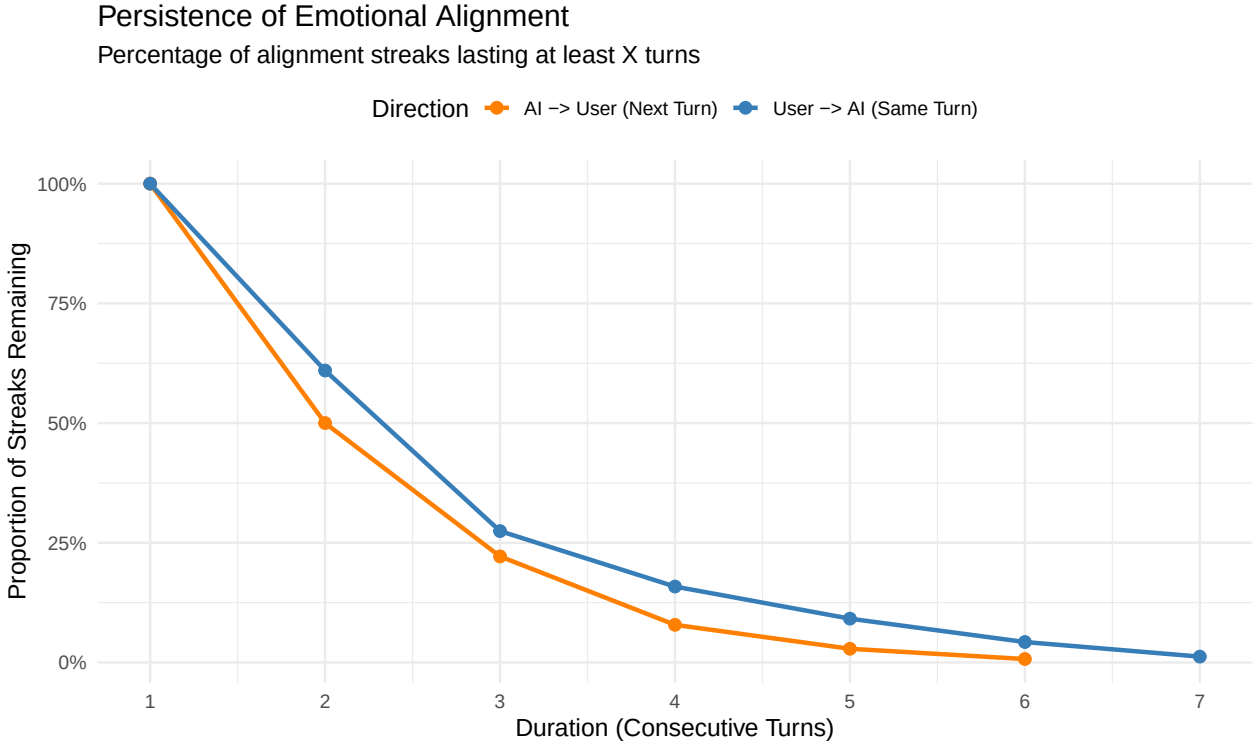
4.12.2 4.7.2 Visualization: Duration Distribution (Survival Analysis)

```r
# Calculate summary stats for durations
duration_stats <- alignment_durations %>%
  group_by(direction_label) %>%
  summarise(
    mean_duration = mean(duration),
    median_duration = median(duration),
    max_duration = max(duration),
    n_streaks = n()
  )
print(duration_stats)
```

```
## # A tibble: 2 x 5
##   direction_label        mean_duration median_duration max_duration n_streaks
##   <chr>                          <dbl>           <dbl>        <int>     <int>
## 1 AI -> User (Next Turn)          1.84             1.5            6       140
## 2 User -> AI (Same Turn)          2.19             2              7       164
```

```r
# Plot: Survival Curve style (Retention of Alignment)
# showing % of streaks that last at least X turns
duration_counts <- alignment_durations %>%
  group_by(direction_label, duration) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(direction_label) %>%
  arrange(desc(duration)) %>%
  mutate(
    cum_count = cumsum(count),
    total_streaks = max(cum_count), # max cumsum is total streaks
    prop_remaining = cum_count / total_streaks
  )

p_duration_curve <- ggplot(duration_counts, aes(x = duration, y = prop_remaining, color = direction_labe
```

```r
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  scale_x_continuous(breaks = 1:10) +
  scale_y_continuous(labels = scales::percent) +
  scale_color_manual(values = c("User -> AI (Same Turn)" = "#377EB8",
                                "AI -> User (Next Turn)" = "#FF7F00")) +
  labs(
    title = "Persistence of Emotional Alignment",
    subtitle = "Percentage of alignment streaks lasting at least X turns",
    x = "Duration (Consecutive Turns)",
    y = "Proportion of Streaks Remaining",
    color = "Direction"
  ) +
  theme(legend.position = "top")

print(p_duration_curve)
```




```r
ggsave(file.path(GRAPHS_DIR, "alignment_duration_survival.png"), p_duration_curve, width = 10, height =

# Statistical Test: Wilcoxon Rank Sum Test for Durations
# (Are streaks in one direction statistically longer?)
wilcox_test <- wilcox.test(duration ~ direction_label, data = alignment_durations)

cat("\n=== Wilcoxon Rank Sum Test: Alignment Duration ===\n")
```

```
##
## === Wilcoxon Rank Sum Test: Alignment Duration ===
```

```r
print(wilcox_test)
```

```
##
```

```
##  Wilcoxon rank sum test with continuity correction
##
## data:  duration by direction_label
## W = 10002, p-value = 0.03943
## alternative hypothesis: true location shift is not equal to 0
```

5 6. Summary and Conclusions

5.1 6.1 Key Findings Summary

```r
# Create summary table
summary_results <- tribble(
  ~Metric, ~Field_Data, ~Simulated_Data,
  "N Stories", n_distinct(df_field$conversation_id), n_distinct(df_simulated$conversation_id),
  "Mean User Valence", mean(df_field$user_valence, na.rm = TRUE), mean(df_simulated$user_valence, na.rm
  "Mean AI Valence", mean(df_field$ai_valence, na.rm = TRUE), mean(df_simulated$ai_valence, na.rm = TRU
  "Same-Turn Alignment ( )", mean(cor_field_same$rho, na.rm = TRUE), mean(cor_sim_same$rho, na.rm = TRUE
  "Previous-Turn Alignment ( )", mean(cor_field_prev$rho, na.rm = TRUE), mean(cor_sim_prev$rho, na.rm =
) %>%
  mutate(
    Field_Data = round(Field_Data, 3),
    Simulated_Data = round(Simulated_Data, 3)
  )

print(summary_results)
```

```
## # A tibble: 5 x 3
##   Metric                      Field_Data Simulated_Data
##   <chr>                            <dbl>          <dbl>
## 1 N Stories                       87              6
## 2 Mean User Valence               -0.089         -0.407
## 3 Mean AI Valence                  0.004         -0.296
## 4 Same-Turn Alignment ( )          0.232         -0.246
## 5 Previous-Turn Alignment ( )      0.091          0.533
```

```r
write_csv(summary_results, file.path(TABLES_DIR, "analysis_summary.csv"))
```

6 7. Supplementary: Interaction Effects

6.1 7.1 Starter Interaction Effects

6.1.1 7.1.1 Same-Turn Alignment by Starter (Removed for now)

```r
lmer_valence_time <- lmer(valence ~ turn + (1 | conversation_id), data = df_field_long)
print(round(summary(lmer_valence_time)$coefficients, 4))
```

```
##             Estimate Std. Error        df t value Pr(>|t|)
## (Intercept)  -0.0389     0.0334  249.4249 -1.1633   0.2458
## turn         -0.0007     0.0040 1304.0000 -0.1724   0.8632
```

```r
# Mixed model with starter interaction
#model_starter_same <- lmer(ai_valence ~ user_valence * starter + (1 | conversation_id),
                            #data = df_field)
```

```
#cat("=== Same-Turn Alignment: Starter Interaction ===\n")
#print(round(summary(model_starter_same)$coefficients, 4))
```

6.1.2 7.1.2 Previous-Turn Alignment by Starter (Removed for now)

```
# Mixed model with starter interaction
#df_field_lag <- df_field %>% filter(!is.na(ai_valence_lag1))

#model_starter_prev <- lmer(user_valence ~ ai_valence_lag1 * starter + (1 | conversation_id),
#                            data = df_field_lag)

#cat("=== Previous-Turn Alignment: Starter Interaction ===\n")
#print(round(summary(model_starter_prev)$coefficients, 4))
```

6.1.3 7.1.3 Per-Story Correlations by Starter (Removed for now)

```
# Same-turn correlations stratified by starter
#cor_starter_same <- df_field %>%
#  filter(!is.na(starter)) %>%
#  group_by(conversation_id, starter) %>%
#  filter(n() >= 3) %>%
#  summarise(
#    rho = cor(user_valence, ai_valence, use = "complete.obs"),
#    n_turns = n(),
#    .groups = "drop"
#  )

#cat("=== Same-Turn Correlations by Starter ===\n")
#cor_starter_same %>%
#  group_by(starter) %>%
#  summarise(
#    mean_rho = mean(rho, na.rm = TRUE),
#    sd_rho = sd(rho, na.rm = TRUE),
#    n_stories = n()
#  ) %>%
#  print()

# Test difference
#t_test_starter <- t.test(rho ~ starter, data = cor_starter_same)
#cat("\nT-test (starter effect on same-turn rho):\n")
#print(t_test_starter)
```

6.2 7.2 LLM Type Interaction Effects

6.2.1 7.2.1 Same-Turn Alignment by LLM Type

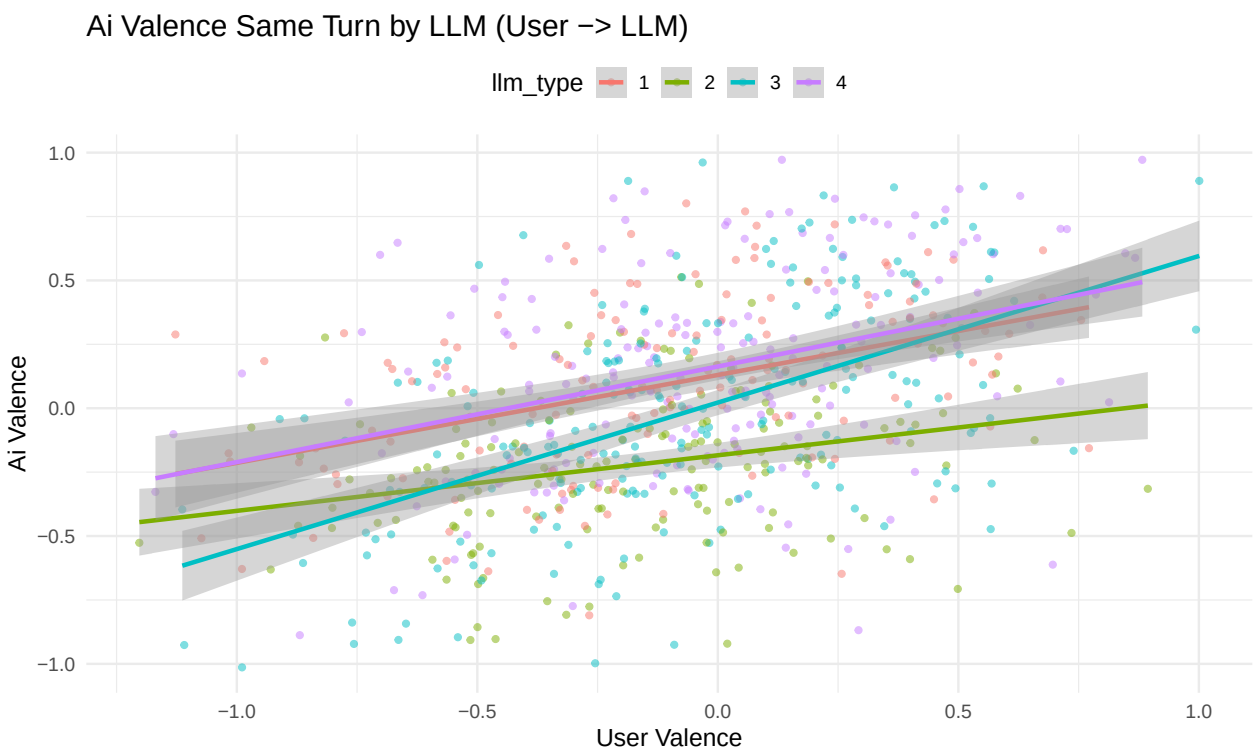
```
# Mixed model with LLM type interaction
df_field_llm <- df_field %>% filter(!is.na(llm_type))

model_llm_same <- lmer(ai_valence ~ user_valence * factor(llm_type) + (1 | conversation_id),
                       data = df_field_llm)

cat("=== Same-Turn Alignment: LLM Type Interaction ===\n")
```

```
## === Same-Turn Alignment: LLM Type Interaction ===
```

```r
print(round(summary(model_llm_same)$coefficients, 4))
```

```
##                                Estimate Std. Error       df t value Pr(>|t|)
## (Intercept)                      0.1208     0.0449  80.1881  2.6891   0.0087
## user_valence                     0.2722     0.0682 676.4800  3.9916   0.0001
## factor(llm_type)2               -0.3146     0.0623  82.2939 -5.0474   0.0000
## factor(llm_type)3               -0.1071     0.0581  78.6317 -1.8434   0.0690
## factor(llm_type)4                0.0399     0.0595  77.9823  0.6709   0.5042
## user_valence:factor(llm_type)2  -0.1164     0.0975 686.8242 -1.1946   0.2327
## user_valence:factor(llm_type)3   0.1684     0.0919 666.1010  1.8313   0.0675
## user_valence:factor(llm_type)4  -0.0032     0.0932 685.0224 -0.0347   0.9723
```

```r
p_llm_same <- df_field_llm %>%
  mutate(llm_type = as.factor(llm_type)) %>%
  ggplot(aes(x = user_valence, y = ai_valence, color = llm_type)) +
  geom_point(alpha = 0.5, size = 1.5) +
  geom_smooth(method = "lm", se = TRUE, size = 1.2) +
  labs(
    title = "Ai Valence Same Turn by LLM (User -> LLM)",
    x = "User Valence",
    y = "Ai Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "top")

print(p_llm_same)
```



```r
ggsave(file.path(GRAPHS_DIR, "alignment_llm_same.png"),
       p_llm_same, width = 12, height = 6, dpi = 300)
```

6.2.2 7.2.2 Previous-Turn Alignment by LLM Type

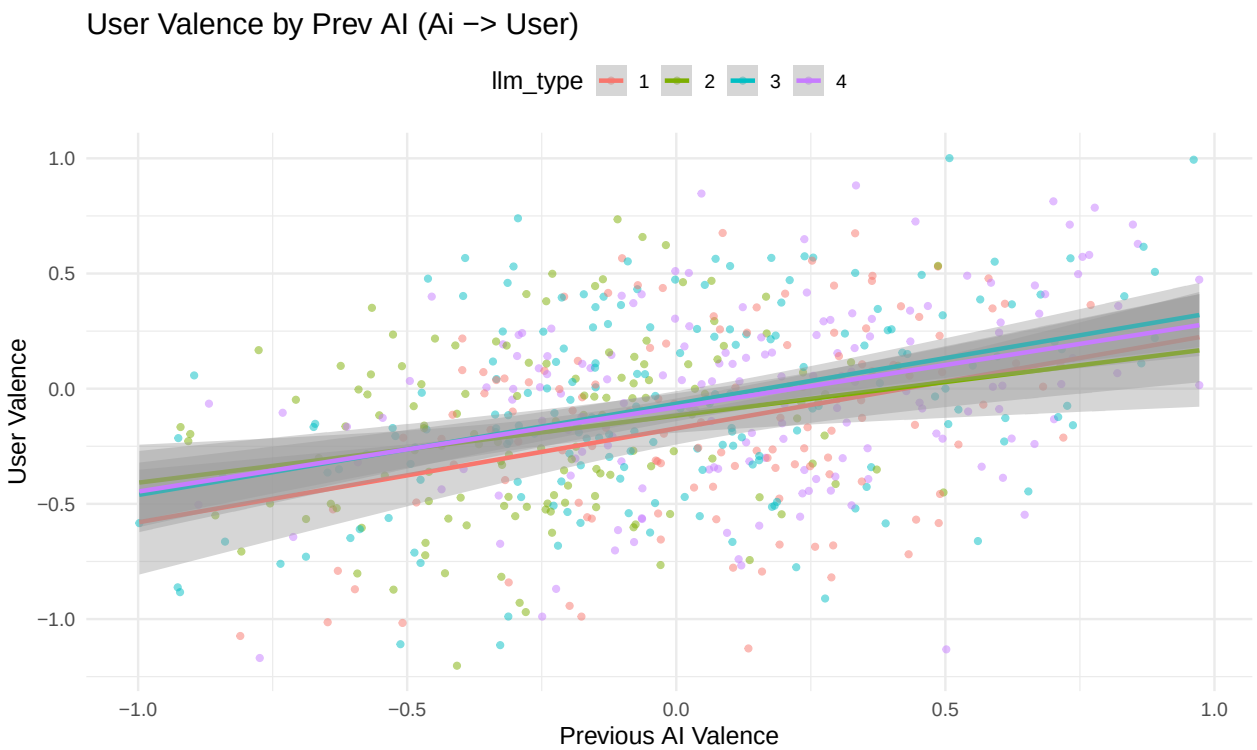
```r
# Mixed model with LLM type interaction
df_field_llm_lag <- df_field %>%
  filter(!is.na(llm_type), !is.na(ai_valence_lag1))

model_llm_prev <- lmer(user_valence ~ ai_valence_lag1 * factor(llm_type) + (1 | conversation_id),
                       data = df_field_llm_lag)

cat("=== Previous-Turn Alignment: LLM Type Interaction ===\n")
```

```
## === Previous-Turn Alignment: LLM Type Interaction ===
```

```r
print(round(summary(model_llm_prev)$coefficients, 4))
```

```
##                                   Estimate Std. Error       df t value Pr(>|t|)
## (Intercept)                        -0.1634     0.0515  74.5484 -3.1731   0.0022
## ai_valence_lag1                     0.2768     0.1031 584.9247  2.6850   0.0075
## factor(llm_type)2                   0.0243     0.0743  88.1319  0.3267   0.7447
## factor(llm_type)3                   0.0962     0.0667  73.6060  1.4412   0.1538
## factor(llm_type)4                   0.0965     0.0694  76.7147  1.3911   0.1682
## ai_valence_lag1:factor(llm_type)2  -0.0849     0.1457 600.0333 -0.5826   0.5604
## ai_valence_lag1:factor(llm_type)3  -0.0596     0.1246 581.6830 -0.4787   0.6323
## ai_valence_lag1:factor(llm_type)4  -0.0047     0.1298 583.9839 -0.0362   0.9711
```

```r
p_llm_prev <- df_field_llm_lag %>%
  mutate(llm_type = as.factor(llm_type)) %>%
  ggplot(aes(x = ai_valence_lag1, y = user_valence, color = llm_type)) +
  geom_point(alpha = 0.5, size = 1.5) +
  geom_smooth(method = "lm", se = TRUE, fullrange= TRUE, size = 1.2) +
  labs(
    title = "User Valence by Prev AI (Ai -> User)",
    x = "Previous AI Valence",
    y = "User Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "top")

print(p_llm_prev)
```




```
ggsave(file.path(GRAPHS_DIR, "alignment_llm_prev.png"),
       p_llm_prev, width = 12, height = 6, dpi = 300)
```

6.2.3 7.2.3 Per-Story Correlations by LLM Type

```
# Same-turn correlations stratified by LLM type
cor_llm_same <- df_field %>%
  filter(!is.na(llm_type)) %>%
  group_by(conversation_id, llm_type) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    .groups = "drop"
  )

cat("=== Same-Turn Correlations by LLM Type ===\n")
```

```
## === Same-Turn Correlations by LLM Type ===
```

```
cor_llm_same %>%
  group_by(llm_type) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 4 x 4
##   llm_type mean_rho sd_rho n_stories
```

```
##      <int>    <dbl>  <dbl>     <int>
## 1        1    0.301  0.353        18
## 2        2    0.198  0.364        20
## 3        3    0.255  0.405        26
## 4        4    0.180  0.432        23
```

```
# ANOVA test
anova_llm <- aov(rho ~ factor(llm_type), data = cor_llm_same)
cat("\nANOVA (LLM type effect on same-turn rho):\n")
```

```
##
## ANOVA (LLM type effect on same-turn rho):
```

```
print(summary(anova_llm))
```

```
##                  Df Sum Sq Mean Sq F value Pr(>F)
## factor(llm_type)  3  0.185 0.06177   0.399  0.754
## Residuals        83 12.846 0.15477
```

6.3 7.3 Valence Distribution by Conditions

6.3.1 7.3.1 Valence by Starter

```
#df_field %>%
#  filter(!is.na(starter)) %>%
#  pivot_longer(cols = c(user_valence, ai_valence),
#               names_to = "agent", values_to = "valence") %>%
#  group_by(starter, agent) %>%
#  summarise(
#    mean_valence = mean(valence, na.rm = TRUE),
#    sd_valence = sd(valence, na.rm = TRUE),
#    .groups = "drop"
#  ) %>%
#  print()
```

6.3.2 7.3.2 Valence by LLM Type

```
df_field %>%
  filter(!is.na(llm_type)) %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  group_by(llm_type, agent) %>%
  summarise(
    mean_valence = mean(valence, na.rm = TRUE),
    sd_valence = sd(valence, na.rm = TRUE),
    n_turns = n(),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 8 x 5
##   llm_type agent        mean_valence sd_valence n_turns
##      <int> <chr>               <dbl>      <dbl>   <int>
## 1        1 ai_valence         0.0854      0.344     144
## 2        1 user_valence      -0.130       0.428     144
```

```
## 3        2 ai_valence        -0.219       0.296     160
## 4        2 user_valence      -0.159       0.381     160
## 5        3 ai_valence        -0.0146      0.425     208
## 6        3 user_valence      -0.0641      0.404     208
## 7        4 ai_valence         0.154       0.394     184
## 8        4 user_valence      -0.0246      0.393     184
```

6.4 8. Session Information

```r
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS 26.3
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Australia/Melbourne
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] here_1.0.2      afex_1.4-1      cowplot_1.1.3   patchwork_1.3.0
##  [5] zoo_1.8-12      stringi_1.8.4   arrow_19.0.1.1  lmerTest_3.1-3
##  [9] lme4_1.1-35.5   Matrix_1.7-1    lubridate_1.9.3 forcats_1.0.0
## [13] stringr_1.5.1   dplyr_1.1.4     purrr_1.0.2     readr_2.1.5
## [17] tidyr_1.3.1     tibble_3.2.1    ggplot2_3.5.1   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1    farver_2.1.2        fastmap_1.2.0
##  [4] TH.data_1.1-2       pacman_0.5.1        digest_0.6.37
##  [7] timechange_0.3.0    estimability_1.5.1  lifecycle_1.0.4
## [10] survival_3.7-0      magrittr_2.0.3      compiler_4.4.2
## [13] rlang_1.1.4         tools_4.4.2         utf8_1.2.4
## [16] yaml_2.3.10         knitr_1.50          labeling_0.4.3
## [19] bit_4.0.5           plyr_1.8.9          abind_1.4-8
## [22] multcomp_1.4-26     withr_3.0.2         numDeriv_2016.8-1.1
## [25] grid_4.4.2          fansi_1.0.6         xtable_1.8-4
## [28] colorspace_2.1-1    emmeans_2.0.1       scales_1.3.0
## [31] MASS_7.3-61         tinytex_0.52        cli_3.6.3
## [34] mvtnorm_1.3-3       rmarkdown_2.28      crayon_1.5.3
## [37] ragg_1.3.2          generics_0.1.3      rstudioapi_0.16.0
## [40] reshape2_1.4.4      tzdb_0.4.0          minqa_1.2.8
## [43] splines_4.4.2       assertthat_0.2.1    parallel_4.4.2
## [46] vctrs_0.6.5         boot_1.3-31         sandwich_3.1-1
## [49] carData_3.0-5       car_3.1-3           hms_1.1.3
## [52] bit64_4.0.5         Formula_1.2-5       systemfonts_1.1.0
```

```
## [55] glue_1.7.0          nloptr_2.1.1        codetools_0.2-20
## [58] gtable_0.3.5        munsell_0.5.1       pillar_1.9.0
## [61] htmltools_0.5.8.1   R6_2.5.1            textshaping_0.4.0
## [64] rprojroot_2.1.1     vroom_1.6.5         evaluate_1.0.3
## [67] lattice_0.22-6      Rcpp_1.0.13         coda_0.19-4.1
## [70] nlme_3.1-166        mgcv_1.9-1          xfun_0.54
## [73] pkgconfig_2.0.3
```

Analysis complete. All figures saved to `analysis/figures/` and tables to `analysis/tables/`.